

Helix AI Full Project Master Report

(In-Depth)

From zero knowledge to fully working system - architecture, code, ML training, UI, testing, and deployment

Generated: 2026-04-05 01:16:33

Goal of this report: give one single complete document that teaches the full project from scratch and explains implementation depth for technical review.

Snapshot Metric	Current Value
Backend files	14
Frontend files	4
ML scripts	12
Backend LOC	1158
Frontend LOC	2647
ML LOC	719
Endpoints	10
Legacy rules	15
Transform branches	15
Unit tests	10 (pass)
Eval pass	4/4 (100.0%)

Chapter 1: Project Foundation

1. Why Legacy Python Modernization Is Needed

Why Legacy Python Modernization Is Needed in simple words: many codebases still contain old syntax that fails on modern runtimes and slows migration. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, why legacy python modernization is needed is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, the analyzer stage detects old patterns before execution. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Understand syntax vs semantic migration issues
- Identify why deterministic migration is safer
- Practice with small legacy snippets first

2. Project Scope and Honesty

Project Scope and Honesty in simple words: we target practical high-impact patterns first, then expand with evidence. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, project scope and honesty is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, scope is explicitly documented in README and architecture docs. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Never claim every historical edge case is auto-fixed
- Publish supported-pattern list clearly
- Use roadmap for unsupported patterns

3. Hybrid Design Choice

Hybrid Design Choice in simple words: rules give safety; ML gives flexibility and future improvement. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, hybrid design choice is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, execution and validation are deterministic while ML is optional. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Know why pure ML can be risky for migration
- Know why pure rules can be limited for ambiguous code
- Explain why hybrid is balanced for student projects

4. What 'Fully Working' Means Here

What 'Fully Working' Means Here in simple words: end-to-end pipeline runs, gives diff/report, and stores run history. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, what 'fully working' means here is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, main.py orchestrates analyze->plan->execute->validate->report->store. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Define completion by behavior, not only UI
- Test endpoints and pipeline logic
- Confirm run history and report generation

Chapter 2: Python and CS Basics Needed (From Scratch)

1. Python Syntax Evolution

Python Syntax Evolution in simple words: Python language changed keywords, builtins, and APIs over versions. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, python syntax evolution is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, legacy rules capture these version-specific differences. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Study print statement to print() migration
- Study xrange/range behavior
- Study dict iterator API changes

2. Parsing and Abstract Syntax Trees

Parsing and Abstract Syntax Trees in simple words: AST is structured representation of code used for analysis. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, parsing and abstract syntax trees is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, analyzer uses `ast.parse` to verify modern parseability and estimate complexity. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Parse sample code with `ast.parse`
- Inspect node types like If/For/Try
- Relate node counts to complexity proxy

3. Regular Expressions for Pattern Detection

Regular Expressions for Pattern Detection in simple words: regex helps quickly detect old syntax patterns in source text. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, regular expressions for pattern detection is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, analyzer and execution use regex for pattern discovery and rewrites. This connection between concept and real code is important because it proves the project is not only

theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Learn raw string regex syntax
- Test pattern false positives
- Use line-aware patterns for syntax rules

4. Deterministic Transformation Pipelines

Deterministic Transformation Pipelines in simple words: deterministic means same input gives same output every time. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, deterministic transformation pipelines is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, planner ordering plus explicit transform ids preserve repeatability. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Keep transform functions pure
- Avoid hidden global state
- Sort candidates deterministically

5. Validation as a Gate

Validation as a Gate in simple words: validation checks are acceptance criteria before output is trusted. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, validation as a gate is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, validation core checks syntax, leftovers, and behavior proxies. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Understand fail-closed behavior
- Separate warnings from hard errors
- Rollback unsafe output

6. Diff-Based Code Review

Diff-Based Code Review in simple words: line-by-line diff makes migration transparent and auditable. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, diff-based code review is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, refactor endpoint returns unified diff and UI diff tab visualizes it. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Read unified diff headers
- Interpret added/removed lines
- Review before apply-to-editor

Chapter 3: System Architecture Deep Dive

1. Analyzer Agent

Analyzer Agent in simple words: finds legacy issues and risk posture before any rewrite. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, analyzer agent is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `agents/analyzer.py` returns issue list, probable version, and risk mode. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Add new legacy patterns with id/message/replacement
- Keep source hints explicit
- Pair each rule with tests

2. Suggestion Agent

Suggestion Agent in simple words: maps detected issues to concrete modernization operations. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, suggestion agent is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `issue-to-suggestion` table controls operation id, confidence, and priority. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Maintain one mapping table
- Include after-hint text
- Keep confidence values justified

3. Critic Agent

Critic Agent in simple words: scores safety and marks warning vs approved status. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, critic agent is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, type-based base scores are adjusted by risk penalties. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented,

testable, and currently running in your local workspace.

What to practice:

- Keep penalty logic simple
- Document thresholds
- Expose reasons in response

4. Planner Agent

Planner Agent in simple words: orders what to execute and filters no-op actions. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, planner agent is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, sorted candidates form `selected_plans` for execution core. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Use stable sort keys
- Keep no-op separated
- Track approved count

5. Execution Core

Execution Core in simple words: applies deterministic operation handlers sequentially. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, execution core is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `core/execution.py` hosts explicit operations for `print`/`xrange`/`raw_input`/and more. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Implement one helper per complex transform
- Preserve newline behavior
- Avoid silent destructive rewrites

6. Validation Core

Validation Core in simple words: decides final pass or fail before output acceptance. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, validation core is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `core/validation.py` enforces syntax/lint/behavior staged checks. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Emit stage-specific errors
- Keep legacy remainder list updated
- Warn on semantic-sensitive areas

7. Reporting + Persistence

Reporting + Persistence in simple words: converts execution context into human-readable and machine-readable artifacts. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, reporting + persistence is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, report generator builds markdown summary and run store writes JSONL history. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Store enough fields for replay
- Summarize list endpoint for performance
- Return `run_id` for UI navigation

8. ML Reasoner Placement

ML Reasoner Placement in simple words: ML path is auxiliary and transparent, not hidden authority. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, ml reasoner placement is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `core/ml_reasoner.py` and `/model-status` expose availability and errors. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Keep deterministic fallback
- Show model status in UI/API
- Gate model outputs by validation in future

Chapter 4: Backend Implementation Walkthrough

1. FastAPI Entry and App Setup

FastAPI Entry and App Setup in simple words: main.py initializes app, templates, static assets, and all agents/cores. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, fastapi entry and app setup is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, module-level singletons wire analyzer/suggester/critic/planner/executor/validator/reporter/run_store/reasoner. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Understand startup wiring
- Track dependency instances
- Keep config centralized

2. Analyze Endpoint Contract

Analyze Endpoint Contract in simple words: analyze is read-only: detect and plan without mutating code. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, analyze endpoint contract is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, returns analysis, suggestions, critiques, plan, and ml status. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Keep analyze idempotent
- Return structured JSON
- Handle failures clearly

3. Refactor Endpoint Contract

Refactor Endpoint Contract in simple words: refactor runs full guarded modernization pipeline. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, refactor endpoint contract is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, returns `run_id`, `new_code`, `diff`, `validation`, `logs`, and `report`. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Rollback if validation fails
- Always produce logs
- Store run record for history

4. Run History Endpoints

Run History Endpoints in simple words: supports list and detail fetch for previous modernization runs. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, run history endpoints is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `run_store` provides summary and full retrieval by id. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Limit list size
- Use `run_id` as key
- Handle missing records with 404

5. Config and Environment

Config and Environment in simple words: env flags control ML availability and adapter paths. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, config and environment is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `core/config.py` centralizes settings and base directories. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Use sane defaults
- Expose toggles for demos
- Keep paths robust on local setups

6. Error Handling and Logging

Error Handling and Logging in simple words: route handlers trap exceptions and return explicit HTTP errors. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering

details so you can both understand and defend the implementation in viva.

From a technical perspective, error handling and logging is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, logs include stage updates for traceable failure analysis. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Catch and surface root cause
- Avoid silent exception swallowing
- Keep logs readable

Chapter 5: Frontend and UX Implementation Walkthrough

1. Single-Page Workflow

Single-Page Workflow in simple words: user can analyze, review, modernize, and validate without leaving page. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, single-page workflow is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `templates/index.html` structures step cards, editor, result tabs, and status badges. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Keep information hierarchy clear
- Avoid abrupt context switches
- Guide next action explicitly

2. State-Driven UI Logic

State-Driven UI Logic in simple words: a central state object powers rendering and action flow. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, state-driven ui logic is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `static/js/app.js` manages `analysis/plan/diff/report/run-history` state. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Update state first, then render
- Avoid direct scattered DOM mutation
- Reset consistently before new runs

3. Primary Action State Machine

Primary Action State Machine in simple words: button behavior changes by workflow phase. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, primary action state machine is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `Analyze -> Modernize -> Apply` logic is handled in one control path. This connection between concept and real code is important because it proves the project is not only

theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Prevent illegal transitions
- Provide clear labels
- Keep keyboard shortcut aligned with primary action

4. Diff-First Review Experience

Diff-First Review Experience in simple words: modernized code is previewed before replacing editor input. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, diff-first review experience is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `renderDiff` and `applyOutputToEditor` preserve user control. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Show diff clearly with colors
- Require explicit apply
- Allow re-analysis after apply

5. Saved Runs UX

Saved Runs UX in simple words: recent runs are clickable and load full report/diff details. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, saved runs ux is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, frontend uses `/runs` and `/runs/{id}` to populate right panel. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Load summaries fast
- Fetch detail on demand
- Keep editor unchanged until user chooses

6. Visual Polish and Accessibility

Visual Polish and Accessibility in simple words: hover states, focus flow, and reduced-motion support improve usability. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, visual polish and accessibility is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `index.html` styles include responsive layout and animation constraints. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Check contrast and readability
- Support keyboard navigation
- Respect reduced-motion preference

Chapter 6: ML Pipeline and Training in Depth

1. ML Goal in This Project

ML Goal in This Project in simple words: model helps modernization reasoning and fallback generation for legacy code. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, ml goal in this project is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, ml folder includes data curation, mixture build, training, inference, and evaluation scripts. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Treat ML as assistant first
- Preserve deterministic authority
- Use evaluation-driven updates

2. Dataset Strategy

Dataset Strategy in simple words: combine curated modernization data with selected public sources.

This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, dataset strategy is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, manifest and builders prepare supervised modernize pairs. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Audit dataset noise
- Keep schema consistent
- Balance mixture weights by task fit

3. Why LoRA + Quantization

Why LoRA + Quantization in simple words: gives practical fine-tuning on affordable GPU budgets. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, why lora + quantization is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, training scripts target Qwen2.5-Coder 1.5B with adapter training path. This connection between concept and real code is important because it proves the project is not only

theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Understand adapter concept
- Track VRAM constraints
- Choose reproducible training config

4. Colab Practical Workflow

Colab Practical Workflow in simple words: upload data, run notebook, download adapter, place in ml/models. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, colab practical workflow is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, COLAB_TRAINING.md documents recommended steps and env flags. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Use GPU runtime
- Archive adapter safely
- Validate model status in app after integration

5. Inference Integration

Inference Integration in simple words: ML reasoner loads base model + adapter and returns generated modernization draft. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, inference integration is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, main API includes model status and optional ML output channel. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Handle missing adapter gracefully
- Expose errors transparently
- Keep response schema stable

6. ML Evaluation and Limits

ML Evaluation and Limits in simple words: small benchmark gives smoke confidence, not full generalization proof. This report keeps the explanation beginner-friendly first, and then moves into deeper engineering details so you can both understand and defend the implementation in viva.

From a technical perspective, ml evaluation and limits is implemented with explicit contracts, deterministic decisions, and visible outputs. This avoids hidden behavior and allows easier debugging when a migration result is not what we expected.

In Helix AI specifically, `evaluate_modernizer.py` provides current rule and optional ML pass stats. This connection between concept and real code is important because it proves the project is not only theoretical; it is implemented, testable, and currently running in your local workspace.

What to practice:

- Report benchmark size always
- Separate rule and ML metrics
- Expand eval set before strong claims

Chapter 7: Mathematical Model and Decision Logic

This chapter explains the formulas used by the current system. The key idea is that each migration candidate gets a safety score and priority-aware ranking before execution.

Critic scoring starts with transform-type base score and applies risk penalties. This keeps the model explainable and helps you justify why one change was marked warning while another was approved.

Planner ranking formula currently follows: $\text{final_score} = \text{safety_score} + \max(0, 50 - \text{priority})$. Candidates are sorted by execution order, then score, then line number to maintain deterministic behavior.

Validation decision is boolean-gated by syntax parse, leftover-legacy lint checks, and behavior proxy checks. If any hard check fails, the pipeline rolls back to original code.

This deterministic math model is intentionally simple because simplicity improves auditability and reduces hidden failure modes in migration workflows.

- $\text{safety_score} = \text{base_score}(\text{type}) - \text{semantic_penalty} - \text{confidence_penalty}$
- $\text{final_score} = \text{safety_score} + \max(0, 50 - \text{priority})$
- $\text{accept_output} = \text{syntax_pass} \text{ AND } \text{lint_pass} \text{ AND } \text{behavior_pass}$
- rollback if $\text{accept_output} == \text{false}$

```
TYPE_BASE_SCORE = {
  'SYNTAX_UPGRADE': 98,
  'API_UPGRADE': 94,
  'TYPE_UPGRADE': 88,
  'SEMANTIC_UPGRADE': 82,
  'NOOP': 100,
}
```

Chapter 8: Testing, Evaluation, and Evidence

Current local test snapshot: status=pass, ran=10 tests.

Current benchmark snapshot: rule_pass=4, benchmark_size=4, pass_rate=100.0 percent.

Testing approach is layered: pipeline tests for transformations, API tests for contract stability, and ML asset checks for data/evaluator readiness.

- Use deterministic legacy fixtures for pipeline tests
- Validate analyze and refactor endpoints separately
- Assert run history retrieval behavior
- Keep benchmark script in CI smoke gate
- Add per-rule regression tests when adding new transforms

Recent test output tail:

```
test_analyze_endpoint_returns_plan_and_ml_status (test_api.ApiTests) ... ok
test_health_and_model_status_endpoints (test_api.ApiTests) ... ok
test_refactor_endpoint_returns_diff_and_validation (test_api.ApiTests) ... ok
test_runs_endpoints_return_stored_history (test_api.ApiTests) ... ok
test_curated_dataset_exists_and_has_examples (test_ml_assets.MLAssetTests) ... ok
test_evaluator_script_runs (test_ml_assets.MLAssetTests) ... ok
test_analyzer_detects_legacy_python_patterns (test_pipeline.PipelineTests) ... ok
test_pipeline_handles_very_old_legacy_patterns (test_pipeline.PipelineTests) ... ok
test_pipeline_modernizes_legacy_code (test_pipeline.PipelineTests) ... ok
test_validation_rejects_leftover_legacy_constructs (test_pipeline.PipelineTests) ... ok
```

```
-----
Ran 10 tests in 0.058s
```

```
OK
```

Chapter 9: Deployment, Operations, and Demo Flow

Local deployment is straightforward: install requirements, run uvicorn, open landing page and inner app routes.

For ML-enabled mode, set environment variables for model enablement, base model, and adapter path. If adapter is absent, app still works with deterministic path.

Operationally, health and model-status endpoints provide fast checks for demo readiness.

Demo flow recommendation: load sample legacy snippet, run analyze, inspect review tab, execute modernization, inspect diff tab, then apply output.

For presentations, keep one blocked-risk sample and one successful migration sample so evaluators can see both safety behavior and happy path.

Run commands:

```
pip install -r requirements.txt
python -m uvicorn main:app --reload

# optional ML mode
export ML_MODEL_ENABLED=true
export ML_MODEL_BASE=Qwen/Qwen2.5-Coder-1.5B-Instruct
export ML_MODEL_ADAPTER_PATH=ml/models/nebula-modernizer-qwen25-1.5b
python -m uvicorn main:app --reload
```

Chapter 10: Full Code Walkthrough Appendix (With Explanation)

This appendix includes key code excerpts plus plain-language and technical explanation for each block, so you can understand the code and explain it confidently.

1. A. main.py (API and orchestration) - Part 1

File: /Users/rutwikbhondave/Desktop/Helix AI Project/main.py

Simple explanation: this code block (lines 1 to 170) mainly handles API orchestration and end-to-end pipeline control. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: API orchestration and end-to-end pipeline control.
- Implementation density in this block: functions=16, classes=2, endpoint decorators=8.
- Key functions to study first: read_landing, health_check, model_status, list_runs, get_run, read_landing, health_check, model_status, list_runs, get_run.
- Key classes in this block: CodeRequest, ContactRequest.
- API surface clues in this block: app.get, app.get, app.get, app.get, app.get, app.get.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import difflib
0002: import json
0003: import logging
0004: import os
0005: from datetime import datetime
0006:
0007: import uvicorn
0008: from fastapi import FastAPI, HTTPException, Request
0009: from fastapi.responses import HTMLResponse, JSONResponse
0010: from fastapi.staticfiles import StaticFiles
0011: from fastapi.templating import Jinja2Templates
0012: from pydantic import BaseModel
0013:
0014: from agents.analyzer import AnalyzerAgent
0015: from agents.critic import CriticAgent
0016: from agents.planner import PlannerAgent
0017: from agents.suggester import SuggestionAgent
0018: from core.config import settings
```

```

0019: from core.execution import ExecutionCore
0020: from core.ml_reasoner import MLReasoner
0021: from core.report_generator import ReportGenerator
0022: from core.run_store import RunStore
0023: from core.validation import ValidationCore
0024:
0025:
0026: analyzer = AnalyzerAgent()
0027: suggester = SuggestionAgent()
0028: critic = CriticAgent()
0029: planner = PlannerAgent()
0030: executor = ExecutionCore()
0031: validator = ValidationCore()
0032: reporter = ReportGenerator()
0033: run_store = RunStore(settings.RUN_HISTORY_FILE)
0034: reasoner = MLReasoner(
0035:     enabled=settings.ML_MODEL_ENABLED,
0036:     adapter_path=os.path.join(settings.BASE_DIR, settings.ML_MODEL_ADAPTER_PATH)
0037:     if not os.path.isabs(settings.ML_MODEL_ADAPTER_PATH)
0038:     else settings.ML_MODEL_ADAPTER_PATH,
0039:     base_model=settings.ML_MODEL_BASE,
0040: )
0041:
0042: app = FastAPI(
0043:     title=settings.APP_NAME,
0044:     version=settings.VERSION,
0045:     docs_url=None,
0046:     redoc_url=None,
0047: )
0048:
0049: app.mount("/static", StaticFiles(directory=settings.STATIC_DIR), name="static")
0050: templates = Jinja2Templates(directory=settings.TEMPLATE_DIR)
0051:
0052:
0053: class CodeRequest(BaseModel):
0054:     code: str
0055:
0056:
0057: class ContactRequest(BaseModel):
0058:     name: str
0059:     email: str
0060:     project: str
0061:     message: str
0062:
0063:
0064: INFO_PAGES = {
0065:     "mission": {
0066:         "title": "Our Mission",
0067:         "content": "<p>Helix AI modernizes legacy Python code with an autonomous multi-agent work
0068:     },
0069:     "team": {
0070:         "title": "The Team",
0071:         "content": "<p>A platform prototype built around agentic planning, rule-based modernization
0072:     },
0073:     "newsletter": {
0074:         "title": "Helix AI Updates",
0075:         "content": "<p>Track new migration rules, supported legacy patterns, and autonomous valida
0076:     },
0077:     "careers": {
0078:         "title": "Join Us",
0079:         "content": "<p>We are focused on developer tooling, code migration, and safe autonomous ex
0080:     },
0081:     "contact": {
0082:         "title": "Contact Us",
0083:         "content": "<p>Questions about legacy Python modernization, agentic workflows, or enterpr
0084:     },
0085:     "refund-policy": {
0086:         "title": "Project Scope",
0087:         "content": "<p>Helix AI focuses on legacy Python modernization. The current prototype str
0088:     },

```

```

0089:     "faqs": {
0090:         "title": "Frequently Asked Questions",
0091:         "content": "<p><strong>Q: What does Helix AI do today?</strong><br>A: It analyzes legacy R
0092:     },
0093:     "stories": {
0094:         "title": "Use Cases",
0095:         "content": "<p>Helix AI is designed for teams modernizing older Python services, education
0096:     },
0097:     "status": {
0098:         "title": "System Status",
0099:         "content": "<p>Modernization pipeline online.</p><ul class='list-disc pl-5 mt-4 space-y-2
0100:     },
0101: }
0102:
0103:
0104: @app.get("/", response_class=HTMLResponse)
0105: async def read_landing(request: Request):
0106:     return templates.TemplateResponse("new.html", {"request": request})
0107:
0108:
0109: @app.get("/health")
0110: async def health_check():
0111:     return JSONResponse(
0112:         content={
0113:             "status": "ok",
0114:             "app": settings.APP_NAME,
0115:             "version": settings.VERSION,
0116:             "ml_reasoner": reasoner.explain_status(),
0117:         }
0118:     )
0119:
0120:
0121: @app.get("/model-status")
0122: async def model_status():
0123:     return JSONResponse(content=reasoner.explain_status())
0124:
0125:
0126: @app.get("/runs")
0127: async def list_runs(limit: int = 10):
0128:     return JSONResponse(content={"runs": run_store.list_runs(limit=max(1, min(limit, 50)))})
0129:
0130:
0131: @app.get("/runs/{run_id}")
0132: async def get_run(run_id: str):
0133:     record = run_store.get_run(run_id)
0134:     if not record:
0135:         raise HTTPException(status_code=404, detail="Run not found")
0136:     return JSONResponse(content=record)
0137:
0138:
0139: @app.get("/app", response_class=HTMLResponse)
0140: async def read_app(request: Request):
0141:     return templates.TemplateResponse("index.html", {"request": request, "app_name": settings.APP
0142:
0143:
0144: @app.get("/info/{page_name}", response_class=HTMLResponse)
0145: async def read_info_page(request: Request, page_name: str):
0146:     page_data = INFO_PAGES.get(page_name)
0147:     if not page_data:
0148:         raise HTTPException(status_code=404, detail="Page not found")
0149:
0150:     return templates.TemplateResponse(
0151:         "info.html",
0152:         {
0153:             "request": request,
0154:             "page_title": page_data["title"],
0155:             "content": page_data["content"],
0156:         },
0157:     )
0158:

```



```
0159:
0160: @app.post("/submit-request")
0161: async def submit_request(request: ContactRequest):
0162:     try:
0163:         logging.info("New Contact Request: %s (%s) - Project: %s", request.name, request.email, request.project)
0164:
0165:         inquiry_data = {
0166:             "name": request.name,
0167:             "email": request.email,
0168:             "project": request.project,
0169:             "message": request.message,
0170:             "timestamp": datetime.now().isoformat(),
```

2. B. main.py (API and orchestration) - Part 2

File: /Users/rutwikbhondave/Desktop/Helix AI Project/main.py

Simple explanation: this code block (lines 171 to 324) mainly handles API orchestration and end-to-end pipeline control. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: API orchestration and end-to-end pipeline control.
- Implementation density in this block: functions=4, classes=0, endpoint decorators=2.
- Key functions to study first: `analyze_code`, `refactor_code`, `analyze_code`, `refactor_code`.
- API surface clues in this block: `app.post`, `app.post`.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0171:         }
0172:
0173:         file_path = "data/inquiries.json"
0174:         inquiries = []
0175:         if os.path.exists(file_path):
0176:             try:
0177:                 with open(file_path, "r", encoding="utf-8") as file:
0178:                     inquiries = json.load(file)
0179:             except Exception as exc:
0180:                 logging.error("Failed to read inquiries: %s", exc)
0181:                 inquiries = []
0182:
0183:         inquiries.append(inquiry_data)
0184:
0185:         with open(file_path, "w", encoding="utf-8") as file:
0186:             json.dump(inquiries, file, indent=4)
0187:
0188:         return JsonResponse(content={"success": True, "message": "Migration inquiry received."})
0189:     except Exception as exc:
0190:         logging.error("Form submission failed: %s", exc)
0191:         raise HTTPException(status_code=500, detail="Transmission failed")
0192:
0193:
0194: @app.post("/analyze")
0195: async def analyze_code(request: CodeRequest):
0196:     try:
0197:         analysis = await analyzer.process(request.code)
0198:         suggestions = await suggester.process(analysis, request.code)
0199:         critiques = await critic.process(suggestions)
```

```

0200:         plan_result = await planner.process(suggestions, critiques)
0201:         ml_status = reasoner.explain_status()
0202:
0203:         return JSONResponse(
0204:             content={
0205:                 "analysis": analysis,
0206:                 "suggestions": suggestions,
0207:                 "critiques": critiques,
0208:                 "plan": plan_result,
0209:                 "ml_reasoner": ml_status,
0210:             }
0211:         )
0212:     except Exception as exc:
0213:         logging.error("Analysis failed: %s", exc)
0214:         raise HTTPException(status_code=500, detail=str(exc))
0215:
0216:
0217: @app.post("/refactor")
0218: async def refactor_code(request: CodeRequest):
0219:     try:
0220:         logs = ["--- Starting Legacy Python Modernization Pipeline ---"]
0221:         run_id = None
0222:
0223:         analysis = await analyzer.process(request.code)
0224:         if not analysis.get("success"):
0225:             return JSONResponse(content={"error": "Analysis failed", "details": analysis.get("error")})
0226:
0227:         logs.append(f"Analyzer: probable source version = {analysis.get('probable_source_version')}")
0228:         logs.append(f"Analyzer: detected {len(analysis.get('legacy_issues', []))} legacy issues.")
0229:
0230:         if analysis.get("mode") == "BLOCKED":
0231:             logs.append("Execution blocked due to critical dynamic execution risk.")
0232:             return JSONResponse(content={"error": "Blocked", "logs": logs, "analysis": analysis})
0233:
0234:         suggestions = await suggester.process(analysis, request.code)
0235:         critique = await critic.process(suggestions)
0236:         plan_result = await planner.process(suggestions, critique)
0237:         selected_plans = plan_result.get("selected_plans", [])
0238:         ml_result = None
0239:
0240:         logs.append(f"Suggester: generated {len(suggestions)} migration suggestions.")
0241:         logs.append(f"Planner: selected {len(selected_plans)} transformations for execution.")
0242:
0243:         new_code = request.code
0244:         validation_result = {"success": False, "stage": "SKIPPED", "message": "No changes applied"}
0245:         validation_success = False
0246:
0247:         if selected_plans:
0248:             new_code = executor.apply_changes(request.code, selected_plans)
0249:             logs.append("Execution: transformations applied sequentially.")
0250:
0251:             validation_result = validator.validate(request.code, new_code)
0252:             if validation_result["success"]:
0253:                 validation_success = True
0254:                 logs.append("Validation: modernization passed all configured checks.")
0255:             else:
0256:                 logs.append(f"Validation failed: {validation_result.get('error')}")
0257:                 logs.append("Rollback: reverting to original code.")
0258:                 new_code = request.code
0259:         else:
0260:             logs.append("Planner: no executable transformations were selected.")
0261:
0262:         if reasoner.is_available():
0263:             ml_result = reasoner.modernize(request.code)
0264:             if ml_result.available and ml_result.output:
0265:                 logs.append("ML Reasoner: generated auxiliary modernization output.")
0266:             elif ml_result.error:
0267:                 logs.append(f"ML Reasoner unavailable: {ml_result.error}")
0268:
0269:         diff = "\n".join(

```

```

0270:         difflib.unified_diff(
0271:             request.code.splitlines(),
0272:             new_code.splitlines(),
0273:             fromfile="legacy.py",
0274:             tofile="modernized.py",
0275:             lineterm="",
0276:         )
0277:     )
0278:
0279:     report = reporter.generate_report(
0280:         analysis=analysis,
0281:         selected_plans=selected_plans,
0282:         success=validation_success,
0283:         validation_logs=logs,
0284:         validation_result=validation_result,
0285:     )
0286:     run_record = run_store.save_run(
0287:         {
0288:             "probable_source_version": analysis.get("probable_source_version"),
0289:             "risk_score": analysis.get("risk_score"),
0290:             "legacy_issue_count": len(analysis.get("legacy_issues", [])),
0291:             "validation_success": validation_success,
0292:             "applied_transformations": [plan["suggestion"]["id"] for plan in selected_plans],
0293:             "original_code": request.code,
0294:             "new_code": new_code,
0295:             "analysis": analysis,
0296:             "validation": validation_result,
0297:             "diff": diff,
0298:             "report": report,
0299:         }
0300:     )
0301:     run_id = run_record["run_id"]
0302:
0303:     return JSONResponse(
0304:         content={
0305:             "run_id": run_id,
0306:             "original_code": request.code,
0307:             "new_code": new_code,
0308:             "analysis": analysis,
0309:             "suggestions": suggestions,
0310:             "plan": plan_result,
0311:             "validation": validation_result,
0312:             "logs": logs,
0313:             "report": report,
0314:             "diff": diff,
0315:             "ml_reasoner": ml_result.__dict__ if ml_result else reasoner.explain_status(),
0316:         }
0317:     )
0318: except Exception as exc:
0319:     logging.error("Refactor failed: %s", exc)
0320:     raise HTTPException(status_code=500, detail=str(exc))
0321:
0322:
0323: if __name__ == "__main__":
0324:     uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

3. C. agents/analyzer.py - Rule detection

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/analyzer.py

Simple explanation: this code block (lines 1 to 220) mainly handles legacy pattern detection, risk scoring mode, and version hinting. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: legacy pattern detection, risk scoring mode, and version hinting.
- Implementation density in this block: functions=5, classes=1, endpoint decorators=0.
- Key functions to study first: `__init__`, `process`, `_parse_python3`, `_find_legacy_issues`, `process`.
- Key classes in this block: `AnalyzerAgent`.
- Regex operations appear here, so correctness depends on pattern precision and edge-case handling.
- AST operations appear here, which usually indicates structured code analysis or validation logic.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Any, Dict, List
0004:
0005: from agents.base_agent import BaseAgent
0006:
0007:
0008: LEGACY_RULES = [
0009:     {
0010:         "id": "backtick_repr",
0011:         "pattern": re.compile(r"`[^`\\n]+`"),
0012:         "source_hint": "Python 1.x/2.x",
0013:         "message": "Backtick repr syntax detected.",
0014:         "replacement": "Replace `expr` with repr(expr).",
0015:         "risk": "MEDIUM",
0016:     },
0017:     {
0018:         "id": "apply_builtin",
0019:         "pattern": re.compile(r"\bapply\s*\("),
0020:         "source_hint": "Python 1.x/2.x",
0021:         "message": "Legacy apply(...) builtin detected.",
0022:         "replacement": "Rewrite apply(fn, args[, kwargs]) as fn(*args[, **kwargs]).",
0023:         "risk": "MEDIUM",
0024:     },
0025:     {
```

```

0026:         "id": "print_statement",
0027:         "pattern": re.compile(r"^\s*print\s+[(\n].*$", re.MULTILINE),
0028:         "source_hint": "Python 2.x",
0029:         "message": "Legacy print statement detected.",
0030:         "replacement": "Wrap values with print(...).",
0031:         "risk": "LOW",
0032:     },
0033:     {
0034:         "id": "xrange_usage",
0035:         "pattern": re.compile(r"\bxrange\s*(\n)",
0036:         "source_hint": "Python 2.x",
0037:         "message": "xrange is not available in modern Python.",
0038:         "replacement": "Replace xrange(...) with range(...).",
0039:         "risk": "LOW",
0040:     },
0041:     {
0042:         "id": "raw_input_usage",
0043:         "pattern": re.compile(r"\braw_input\s*(\n)",
0044:         "source_hint": "Python 2.x",
0045:         "message": "raw_input was renamed in Python 3.",
0046:         "replacement": "Replace raw_input(...) with input(...).",
0047:         "risk": "LOW",
0048:     },
0049:     {
0050:         "id": "except_comma",
0051:         "pattern": re.compile(r"^\s*except\s+[(\n]+\s*[A-Za-z_]\w*\s*:", re.MULTILINE),
0052:         "source_hint": "Python 2.x",
0053:         "message": "Old exception binding syntax detected.",
0054:         "replacement": "Convert except X, e: to except X as e:.",
0055:         "risk": "LOW",
0056:     },
0057:     {
0058:         "id": "not_equal_operator",
0059:         "pattern": re.compile(r"<>"),
0060:         "source_hint": "Python 2.x",
0061:         "message": "Legacy not-equal operator detected.",
0062:         "replacement": "Replace <> with !=",
0063:         "risk": "LOW",
0064:     },
0065:     {
0066:         "id": "iteritems_usage",
0067:         "pattern": re.compile(r"\.iteritems\s*(\n)",
0068:         "source_hint": "Python 2.x",
0069:         "message": "iteritems returns a Python 2 iterator API.",
0070:         "replacement": "Replace iteritems() with items().",
0071:         "risk": "LOW",
0072:     },
0073:     {
0074:         "id": "iterkeys_usage",
0075:         "pattern": re.compile(r"\.iterkeys\s*(\n)",
0076:         "source_hint": "Python 2.x",
0077:         "message": "iterkeys returns a Python 2 iterator API.",
0078:         "replacement": "Replace iterkeys() with keys().",
0079:         "risk": "LOW",
0080:     },
0081:     {
0082:         "id": "intervalues_usage",
0083:         "pattern": re.compile(r"\.intervalues\s*(\n)",
0084:         "source_hint": "Python 2.x",
0085:         "message": "intervalues returns a Python 2 iterator API.",
0086:         "replacement": "Replace intervalues() with values().",
0087:         "risk": "LOW",
0088:     },
0089:     {
0090:         "id": "has_key_usage",
0091:         "pattern": re.compile(r"\.has_key\s*(\n)",
0092:         "source_hint": "Python 2.x",
0093:         "message": "dict.has_key was removed.",
0094:         "replacement": "Rewrite d.has_key(x) as x in d.",
0095:         "risk": "MEDIUM",

```

```

0096:     },
0097:     {
0098:         "id": "unicode_type",
0099:         "pattern": re.compile(r"\bunicode\b"),
0100:         "source_hint": "Python 2.x",
0101:         "message": "unicode type detected.",
0102:         "replacement": "Replace unicode with str where compatible.",
0103:         "risk": "MEDIUM",
0104:     },
0105:     {
0106:         "id": "basestring_type",
0107:         "pattern": re.compile(r"\bbasestring\b"),
0108:         "source_hint": "Python 2.x",
0109:         "message": "basestring type detected.",
0110:         "replacement": "Replace basestring with str.",
0111:         "risk": "MEDIUM",
0112:     },
0113:     {
0114:         "id": "long_type",
0115:         "pattern": re.compile(r"\blong\b"),
0116:         "source_hint": "Python 2.x",
0117:         "message": "long type detected.",
0118:         "replacement": "Replace long with int.",
0119:         "risk": "LOW",
0120:     },
0121:     {
0122:         "id": "exec_statement",
0123:         "pattern": re.compile(r"^\s*exec\s+(\^\(\n\).*$", re.MULTILINE),
0124:         "source_hint": "Python 1.x/2.x",
0125:         "message": "Legacy exec statement detected.",
0126:         "replacement": "Convert exec statement syntax to exec(...).",
0127:         "risk": "HIGH",
0128:     },
0129: ]
0130:
0131:
0132: HIGH_RISK_PATTERNS = [
0133:     {
0134:         "id": "division_semantics",
0135:         "pattern": re.compile(r"(?!/)(?!/)"),
0136:         "message": "Division semantics may change between Python 2 and Python 3.",
0137:         "risk": "HIGH",
0138:     },
0139:     {
0140:         "id": "bytes_text_boundary",
0141:         "pattern": re.compile(r"\b(str|unicode|bytes)\b"),
0142:         "message": "Text/bytes handling may require manual review.",
0143:         "risk": "MEDIUM",
0144:     },
0145: ]
0146:
0147:
0148: class AnalyzerAgent(BaseAgent):
0149:     def __init__(self):
0150:         super().__init__("Analyzer")
0151:
0152:     async def process(self, code: str) -> Dict[str, Any]:
0153:         issues = self._find_legacy_issues(code)
0154:         risks = self._find_semantic_risks(code)
0155:         probable_version = self._infer_version(issues)
0156:         parse_result = self._parse_python3(code)
0157:
0158:         complexity = parse_result.get("complexity", 0)
0159:         parseable = parse_result["parseable"]
0160:         parse_error = parse_result.get("error")
0161:
0162:         risk_score = "LOW"
0163:         mode = "SAFE"
0164:         risk_messages: List[str] = [risk["message"] for risk in risks]
0165:

```

```

0166:         if issues:
0167:             risk_score = "MEDIUM"
0168:             mode = "RESTRICTED"
0169:
0170:         if any(risk["risk"] == "HIGH" for risk in risks):
0171:             risk_score = "HIGH"
0172:             mode = "RESTRICTED"
0173:
0174:         if parse_error and not issues:
0175:             return {
0176:                 "success": False,
0177:                 "error": parse_error,
0178:                 "parseable": False,
0179:             }
0180:
0181:         if re.search(r"\b(eval|exec)\s*(", code):
0182:             risk_score = "CRITICAL"
0183:             mode = "BLOCKED"
0184:             risk_messages.append("Detected unsafe dynamic execution.")
0185:
0186:         return {
0187:             "success": True,
0188:             "loc": len(code.splitlines()),
0189:             "complexity": complexity,
0190:             "risk_score": risk_score,
0191:             "mode": mode,
0192:             "parseable": parseable,
0193:             "parse_error": parse_error,
0194:             "probable_source_version": probable_version,
0195:             "legacy_issues": issues,
0196:             "semantic_risks": risks,
0197:             "risks": risk_messages,
0198:             "ast_dump": parse_result.get("ast_dump"),
0199:         }
0200:
0201:     def _parse_python3(self, code: str) -> Dict[str, Any]:
0202:         try:
0203:             tree = ast.parse(code)
0204:         except SyntaxError as exc:
0205:             return {"parseable": False, "error": str(exc)}
0206:
0207:         complexity = 1
0208:         for node in ast.walk(tree):
0209:             if isinstance(node, (ast.If, ast.For, ast.While, ast.ExceptHandler, ast.With, ast.Try)):
0210:                 complexity += 1
0211:
0212:         return {
0213:             "parseable": True,
0214:             "complexity": complexity,
0215:             "ast_dump": ast.dump(tree),
0216:         }
0217:
0218:     def _find_legacy_issues(self, code: str) -> List[Dict[str, Any]]:
0219:         lines = code.splitlines()
0220:         issues: List[Dict[str, Any]] = []

```


4. D. agents/suggester.py - Mapping table

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/suggester.py

Simple explanation: this code block (lines 1 to 192) mainly handles mapping detected issues to concrete modernization actions. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: mapping detected issues to concrete modernization actions.
- Implementation density in this block: functions=3, classes=1, endpoint decorators=0.
- Key functions to study first: `__init__`, `process`, `process`.
- Key classes in this block: `SuggestionAgent`.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: from typing import Any, Dict, List
0002:
0003: from agents.base_agent import BaseAgent
0004:
0005:
0006: ISSUE_TO_SUGGESTION = {
0007:     "backtick_repr": {
0008:         "id": "upgrade_backtick_repr",
0009:         "type": "SEMANTIC_UPGRADE",
0010:         "target": "backtick repr",
0011:         "suggestion": "repr_call",
0012:         "reasoning": "Backticks were an old repr shorthand and should become repr(...).",
0013:         "confidence": 0.9,
0014:         "priority": 12,
0015:     },
0016:     "apply_builtin": {
0017:         "id": "upgrade_apply_builtin",
0018:         "type": "SEMANTIC_UPGRADE",
0019:         "target": "apply builtin",
0020:         "suggestion": "star_call",
0021:         "reasoning": "apply(fn, args[, kwargs]) is now expressed as fn(*args[, **kwargs]).",
0022:         "confidence": 0.9,
0023:         "priority": 18,
0024:     },
0025:     "print_statement": {
0026:         "id": "upgrade_print_statement",
0027:         "type": "SYNTAX_UPGRADE",
0028:         "target": "print statement",
0029:         "suggestion": "print_function",
```

```

0030:         "reasoning": "Convert legacy print statements to Python 3 print(...) calls.",
0031:         "confidence": 0.98,
0032:         "priority": 10,
0033:     },
0034:     "xrange_usage": {
0035:         "id": "upgrade_xrange",
0036:         "type": "API_UPGRADE",
0037:         "target": "xrange",
0038:         "suggestion": "range",
0039:         "reasoning": "xrange was removed; range provides the modern equivalent.",
0040:         "confidence": 0.99,
0041:         "priority": 20,
0042:     },
0043:     "raw_input_usage": {
0044:         "id": "upgrade_raw_input",
0045:         "type": "API_UPGRADE",
0046:         "target": "raw_input",
0047:         "suggestion": "input",
0048:         "reasoning": "raw_input was renamed to input in Python 3.",
0049:         "confidence": 0.99,
0050:         "priority": 20,
0051:     },
0052:     "except_comma": {
0053:         "id": "upgrade_except_syntax",
0054:         "type": "SYNTAX_UPGRADE",
0055:         "target": "except syntax",
0056:         "suggestion": "except as",
0057:         "reasoning": "Modern Python binds exceptions with `as`.",
0058:         "confidence": 0.97,
0059:         "priority": 15,
0060:     },
0061:     "not_equal_operator": {
0062:         "id": "upgrade_not_equal",
0063:         "type": "SYNTAX_UPGRADE",
0064:         "target": "<>",
0065:         "suggestion": "!=",
0066:         "reasoning": "The <> operator is obsolete.",
0067:         "confidence": 1.0,
0068:         "priority": 10,
0069:     },
0070:     "iteritems_usage": {
0071:         "id": "upgrade_iteritems",
0072:         "type": "API_UPGRADE",
0073:         "target": "iteritems",
0074:         "suggestion": "items",
0075:         "reasoning": "Use items() for modern dictionary iteration.",
0076:         "confidence": 0.98,
0077:         "priority": 25,
0078:     },
0079:     "iterkeys_usage": {
0080:         "id": "upgrade_iterkeys",
0081:         "type": "API_UPGRADE",
0082:         "target": "iterkeys",
0083:         "suggestion": "keys",
0084:         "reasoning": "Use keys() for modern dictionary iteration.",
0085:         "confidence": 0.98,
0086:         "priority": 25,
0087:     },
0088:     "itervalues_usage": {
0089:         "id": "upgrade_itervalues",
0090:         "type": "API_UPGRADE",
0091:         "target": "itervalues",
0092:         "suggestion": "values",
0093:         "reasoning": "Use values() for modern dictionary iteration.",
0094:         "confidence": 0.98,
0095:         "priority": 25,
0096:     },
0097:     "has_key_usage": {
0098:         "id": "upgrade_has_key",
0099:         "type": "SEMANTIC_UPGRADE",

```

```

0100:         "target": "has_key",
0101:         "suggestion": "in_operator",
0102:         "reasoning": "Replace deprecated has_key checks with membership tests.",
0103:         "confidence": 0.92,
0104:         "priority": 30,
0105:     },
0106:     "unicode_type": {
0107:         "id": "upgrade_unicode",
0108:         "type": "TYPE_UPGRADE",
0109:         "target": "unicode",
0110:         "suggestion": "str",
0111:         "reasoning": "Python 3 uses str for text values.",
0112:         "confidence": 0.88,
0113:         "priority": 35,
0114:     },
0115:     "basestring_type": {
0116:         "id": "upgrade_basestring",
0117:         "type": "TYPE_UPGRADE",
0118:         "target": "basestring",
0119:         "suggestion": "str",
0120:         "reasoning": "basestring no longer exists in Python 3.",
0121:         "confidence": 0.9,
0122:         "priority": 35,
0123:     },
0124:     "long_type": {
0125:         "id": "upgrade_long",
0126:         "type": "TYPE_UPGRADE",
0127:         "target": "long",
0128:         "suggestion": "int",
0129:         "reasoning": "int subsumes long in modern Python.",
0130:         "confidence": 0.96,
0131:         "priority": 35,
0132:     },
0133:     "exec_statement": {
0134:         "id": "upgrade_exec_statement",
0135:         "type": "SEMANTIC_UPGRADE",
0136:         "target": "exec statement",
0137:         "suggestion": "exec_function",
0138:         "reasoning": "Modern Python uses exec(...) function-style syntax.",
0139:         "confidence": 0.86,
0140:         "priority": 22,
0141:     },
0142: }
0143:
0144:
0145: class SuggestionAgent(BaseAgent):
0146:     def __init__(self):
0147:         super().__init__("Suggester")
0148:
0149:     async def process(self, analysis_result: Dict[str, Any], code: str) -> List[Dict[str, Any]]:
0150:         if not analysis_result.get("success"):
0151:             return []
0152:
0153:         suggestions: List[Dict[str, Any]] = []
0154:         for issue in analysis_result.get("legacy_issues", []):
0155:             base = ISSUE_TO_SUGGESTION.get(issue["id"])
0156:             if not base:
0157:                 continue
0158:
0159:             suggestion = dict(base)
0160:             suggestion.update(
0161:                 {
0162:                     "line": issue["line"],
0163:                     "preview": issue["preview"],
0164:                     "risk": issue["risk"],
0165:                     "source_hint": issue["source_hint"],
0166:                     "before": issue["preview"],
0167:                     "after_hint": issue["replacement"],
0168:                 }
0169:             )

```

```

0170:         suggestions.append(suggestion)
0171:
0172:     # Provide at least one actionable path for already-modern code.
0173:     if not suggestions:
0174:         suggestions.append(
0175:             {
0176:                 "id": "no_migration_needed",
0177:                 "type": "NOOP",
0178:                 "target": "codebase",
0179:                 "suggestion": "already_modern",
0180:                 "reasoning": "No high-confidence legacy Python patterns were detected.",
0181:                 "confidence": 0.95,
0182:                 "priority": 100,
0183:                 "line": 1,
0184:                 "preview": code.splitlines()[0] if code.splitlines() else "",
0185:                 "risk": "LOW",
0186:                 "source_hint": "Python 3.x",
0187:                 "before": "",
0188:                 "after_hint": "No migration changes required.",
0189:             }
0190:         )
0191:
0192:     return suggestions

```

5. E. agents/critic.py - Safety scoring

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/critic.py

Simple explanation: this code block (lines 1 to 51) mainly handles safety scoring and warning classification for suggestions. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: safety scoring and warning classification for suggestions.
- Implementation density in this block: functions=3, classes=1, endpoint decorators=0.
- Key functions to study first: `__init__`, `process`, `process`.
- Key classes in this block: `CriticAgent`.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: from typing import Any, Dict, List
0002:
0003: from agents.base_agent import BaseAgent
0004:
0005:
0006: TYPE_BASE_SCORE = {
0007:     "SYNTAX_UPGRADE": 98,
0008:     "API_UPGRADE": 94,
0009:     "TYPE_UPGRADE": 88,
0010:     "SEMANTIC_UPGRADE": 82,
0011:     "NOOP": 100,
0012: }
0013:
0014:
0015: class CriticAgent(BaseAgent):
0016:     def __init__(self):
0017:         super().__init__("Critic")
0018:
0019:     async def process(self, suggestions: List[Dict[str, Any]]) -> Dict[str, Any]:
0020:         critique_results = []
0021:
0022:         for suggestion in suggestions:
0023:             score = TYPE_BASE_SCORE.get(suggestion["type"], 75)
0024:             status = "APPROVED"
0025:             reasons = []
0026:
0027:             if suggestion["type"] == "NOOP":
0028:                 reasons.append("No migration action needed.")
0029:             if suggestion["type"] == "SEMANTIC_UPGRADE":
0030:                 status = "WARNING"
0031:                 score -= 10
```

```

0032:         reasons.append("Semantic rewrite requires careful validation.")
0033:     if suggestion["type"] == "TYPE_UPGRADE":
0034:         status = "WARNING"
0035:         score -= 5
0036:         reasons.append("Type migration may affect string/bytes behavior.")
0037:     if suggestion["confidence"] < 0.9:
0038:         status = "WARNING"
0039:         score -= 5
0040:         reasons.append("Confidence is below strict auto-apply threshold.")
0041:
0042:     critique_results.append(
0043:         {
0044:             "suggestion_id": suggestion["id"],
0045:             "status": status,
0046:             "safety_score": max(score, 0),
0047:             "reasons": reasons,
0048:         }
0049:     )
0050:
0051: return {"critiques": critique_results}

```

6. F. agents/planner.py - Candidate ordering

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/planner.py

Simple explanation: this code block (lines 1 to 40) mainly handles execution ordering and candidate selection. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: execution ordering and candidate selection.
- Implementation density in this block: functions=3, classes=1, endpoint decorators=0.
- Key functions to study first: `__init__`, `process`, `process`.
- Key classes in this block: `PlannerAgent`.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: from typing import Any, Dict, List
0002:
0003: from agents.base_agent import BaseAgent
0004:
0005:
0006: class PlannerAgent(BaseAgent):
0007:     def __init__(self):
0008:         super().__init__("Planner")
0009:
0010:     async def process(self, suggestions: List[Dict[str, Any]], critiques: Dict[str, Any]) -> Dict:
0011:         critique_map = {item["suggestion_id"]: item for item in critiques["critiques"]}
0012:         candidates = []
0013:
0014:         for suggestion in suggestions:
0015:             critique = critique_map.get(suggestion["id"])
0016:             if not critique:
0017:                 continue
0018:             if critique["status"] not in {"APPROVED", "WARNING"}:
0019:                 continue
0020:
0021:             final_score = critique["safety_score"] + max(0, 50 - suggestion.get("priority", 50))
0022:             candidates.append(
0023:                 {
0024:                     "suggestion": suggestion,
0025:                     "critique": critique,
0026:                     "safety_score": final_score,
0027:                     "execution_order": suggestion.get("priority", 50),
0028:                 }
0029:             )
0030:
0031:             candidates.sort(key=lambda item: (item["execution_order"], -item["safety_score"], item["suggestion_id"])
```

```
0032:         selected_plans = [item for item in candidates if item["suggestion"]["type"] != "NOOP"]
0033:
0034:     return {
0035:         "selected_plan": selected_plans[0] if selected_plans else None,
0036:         "selected_plans": selected_plans,
0037:         "total_candidates": len(suggestions),
0038:         "approved_count": len(selected_plans),
0039:         "candidates": candidates,
0040:     }
```


7. G. core/execution.py - Transform engine

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/execution.py

Simple explanation: this code block (lines 1 to 114) mainly handles deterministic code transformation engine. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: deterministic code transformation engine.
- Implementation density in this block: functions=7, classes=1, endpoint decorators=0.
- Key functions to study first: `apply_changes`, `_apply_single`, `_upgrade_print_statements`, `_upgrade_backtick_repr`, `_upgrade_apply_builtin`.
- Key classes in this block: `ExecutionCore`.
- Regex operations appear here, so correctness depends on pattern precision and edge-case handling.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import re
0002: from typing import List
0003:
0004:
0005: class ExecutionCore:
0006:     def apply_changes(self, code: str, plans) -> str:
0007:         if not plans:
0008:             return code
0009:
0010:         if isinstance(plans, dict):
0011:             plans = [plans]
0012:
0013:         candidate = code
0014:         for plan in plans:
0015:             candidate = self._apply_single(candidate, plan["suggestion"] if "suggestion" in plan else plan["operation"])
0016:         return candidate
0017:
0018:     def _apply_single(self, code: str, suggestion: dict) -> str:
0019:         op = suggestion["id"]
0020:
0021:         if op == "upgrade_print_statement":
0022:             return self._upgrade_print_statements(code)
0023:         if op == "upgrade_backtick_repr":
0024:             return self._upgrade_backtick_repr(code)
0025:         if op == "upgrade_apply_builtin":
0026:             return self._upgrade_apply_builtin(code)
0027:         if op == "upgrade_xrange":
0028:             return re.sub(r"\bxrange(?\s*\()", "range", code)
0029:         if op == "upgrade_raw_input":
0030:             return re.sub(r"\braw_input(?\s*\()", "input", code)
```

```

0031:         if op == "upgrade_except_syntax":
0032:             return re.sub(r"except\s+([\^:\n]+),\s*([A-Za-z_]\w*)\s*:", r"except \1 as \2:", code)
0033:         if op == "upgrade_exec_statement":
0034:             return self._upgrade_exec_statement(code)
0035:         if op == "upgrade_not_equal":
0036:             return code.replace("<", "!=")
0037:         if op == "upgrade_iteritems":
0038:             return re.sub(r"\.iteritems(?\s*\()", ".items", code)
0039:         if op == "upgrade_iterkeys":
0040:             return re.sub(r"\.iterkeys(?\s*\()", ".keys", code)
0041:         if op == "upgrade_intervalvalues":
0042:             return re.sub(r"\.intervalvalues(?\s*\()", ".values", code)
0043:         if op == "upgrade_has_key":
0044:             return re.sub(r"([A-Za-z_][\w\.]*)\.has_key\(((\^)+)\)", r"\2 in \1", code)
0045:         if op == "upgrade_unicode":
0046:             return re.sub(r"\bunicode\b", "str", code)
0047:         if op == "upgrade_basestring":
0048:             return re.sub(r"\bbasestring\b", "str", code)
0049:         if op == "upgrade_long":
0050:             return re.sub(r"\blong\b", "int", code)
0051:         return code
0052:
0053:     def _upgrade_print_statements(self, code: str) -> str:
0054:         upgraded_lines: List[str] = []
0055:         for line in code.splitlines():
0056:             stripped = line.lstrip()
0057:             indent = line[: len(line) - len(stripped)]
0058:
0059:             if not stripped.startswith("print ") or stripped.startswith("print ("):
0060:                 upgraded_lines.append(line)
0061:                 continue
0062:
0063:             if stripped.startswith("print >>"):
0064:                 upgraded_lines.append(f"{indent}# TODO: manual migration required for redirected p
0065:                 continue
0066:
0067:             payload = stripped[6:].rstrip()
0068:             if payload.endswith(","):
0069:                 payload = payload[:-1].rstrip()
0070:             upgraded_lines.append(f"{indent}print({payload})")
0071:
0072:         return "\n".join(upgraded_lines) + ("\n" if code.endswith("\n") else "")
0073:
0074:     def _upgrade_backtick_repr(self, code: str) -> str:
0075:         return re.sub(r"``([\^:\n]+)`", r"repr(\1)", code)
0076:
0077:     def _upgrade_apply_builtin(self, code: str) -> str:
0078:         def replace(match):
0079:             fn_name = match.group(1).strip()
0080:             args_expr = match.group(2).strip()
0081:             kwargs_expr = match.group(3)
0082:             if kwargs_expr:
0083:                 return f"{fn_name}({args_expr}, **{kwargs_expr.strip()})"
0084:             return f"{fn_name}({args_expr})"
0085:
0086:         pattern = re.compile(
0087:             r"\bapply\s*\(\s*([A-Za-z_][\w\.]*)\s*,\s*([\^,\n][\^,\n]*)\s*(?:,\s*([\^]+?)\s*)?\s*\)"
0088:         )
0089:         return pattern.sub(replace, code)
0090:
0091:     def _upgrade_exec_statement(self, code: str) -> str:
0092:         upgraded_lines: List[str] = []
0093:         for line in code.splitlines():
0094:             stripped = line.lstrip()
0095:             indent = line[: len(line) - len(stripped)]
0096:
0097:             if not stripped.startswith("exec ") or stripped.startswith("exec("):
0098:                 upgraded_lines.append(line)
0099:                 continue
0100:

```

```

0101:         payload = stripped[5:].strip()
0102:         if " in " in payload:
0103:             expr, env = payload.split(" in ", 1)
0104:             parts = [part.strip() for part in env.split(",")]
0105:             if len(parts) == 2:
0106:                 upgraded_lines.append(f"{indent}exec({expr.strip()}, {parts[0]}, {parts[1]})")
0107:                 continue
0108:             if len(parts) == 1 and parts[0]:
0109:                 upgraded_lines.append(f"{indent}exec({expr.strip()}, {parts[0]})")
0110:                 continue
0111:
0112:         upgraded_lines.append(f"{indent}exec({payload})")
0113:
0114:     return "\n".join(upgraded_lines) + ("\n" if code.endswith("\n") else "")

```

8. H. core/validation.py - Safety gate

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/validation.py

Simple explanation: this code block (lines 1 to 100) mainly handles acceptance gate for migrated output. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: acceptance gate for migrated output.
- Implementation density in this block: functions=4, classes=1, endpoint decorators=0.
- Key functions to study first: `validate`, `_check_lint`, `_verify_behavior`, `_get_function_names`.
- Key classes in this block: `ValidationCore`.
- Regex operations appear here, so correctness depends on pattern precision and edge-case handling.
- AST operations appear here, which usually indicates structured code analysis or validation logic.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Dict, Any, List
0004:
0005:
0006: LEGACY_REMAINDER_PATTERNS = {
0007:     "backtick_repr": re.compile(r"``[\n]+`"),
0008:     "apply_builtin": re.compile(r"\bapply\s*\(",
0009:     "xrange": re.compile(r"\bxrange\s*\((",
0010:     "raw_input": re.compile(r"\braw_input\s*\((",
0011:     "iteritems": re.compile(r"\.iteritems\s*\((",
0012:     "iterkeys": re.compile(r"\.iterkeys\s*\((",
0013:     "itervalues": re.compile(r"\.itervalues\s*\((",
0014:     "has_key": re.compile(r"\.has_key\s*\((",
0015:     "not_equal": re.compile(r"<>"),
0016:     "except_comma": re.compile(r"except\s+([\n:]+),\s*([A-Za-z_]\w*)\s*:"),
0017:     "exec_statement": re.compile(r"^\s*exec\s+[\n].*$", re.MULTILINE),
0018: }
0019:
0020:
0021: class ValidationCore:
0022:     def validate(self, original_code: str, new_code: str) -> Dict[str, Any]:
0023:         warnings: List[str] = []
0024:
0025:         try:
0026:             ast.parse(new_code)
0027:         except SyntaxError as exc:
```

```

0028:         return {
0029:             "success": False,
0030:             "stage": "SYNTAX",
0031:             "error": f"Syntax Error: {exc.msg} at line {exc.lineno}",
0032:             "warnings": warnings,
0033:         }
0034:
0035:     lint_result = self._check_lint(new_code)
0036:     warnings.extend(lint_result.get("warnings", []))
0037:     if not lint_result["success"]:
0038:         return {
0039:             "success": False,
0040:             "stage": "LINT",
0041:             "error": lint_result["error"],
0042:             "warnings": warnings,
0043:         }
0044:
0045:     behavior_result = self._verify_behavior(original_code, new_code)
0046:     warnings.extend(behavior_result.get("warnings", []))
0047:     if not behavior_result["success"]:
0048:         return {
0049:             "success": False,
0050:             "stage": "BEHAVIOR",
0051:             "error": behavior_result["error"],
0052:             "warnings": warnings,
0053:         }
0054:
0055:     return {
0056:         "success": True,
0057:         "stage": "COMPLETE",
0058:         "message": "Modernization validation passed.",
0059:         "warnings": warnings,
0060:     }
0061:
0062: def _check_lint(self, code: str) -> Dict[str, Any]:
0063:     warnings: List[str] = []
0064:     leftovers = [name for name, pattern in LEGACY_REMAINDER_PATTERNS.items() if pattern.search(
0065:     if leftovers:
0066:         return {
0067:             "success": False,
0068:             "error": f"Legacy constructs still remain after migration: {'', '.join(leftovers)}",
0069:             "warnings": warnings,
0070:         }
0071:
0072:     if re.search(r"(?<!/)/(?!/)", code):
0073:         warnings.append("Division operators remain; review Python 2 to 3 semantics manually.")
0074:
0075:     return {"success": True, "warnings": warnings}
0076:
0077: def _verify_behavior(self, original: str, new: str) -> Dict[str, Any]:
0078:     warnings: List[str] = []
0079:     original_functions = self._get_function_names(original)
0080:     new_functions = self._get_function_names(new)
0081:
0082:     missing = original_functions - new_functions
0083:     if missing:
0084:         return {
0085:             "success": False,
0086:             "error": f"Original functions disappeared during migration: {sorted(missing)}",
0087:             "warnings": warnings,
0088:         }
0089:
0090:     if "unicode" in original or "basestring" in original:
0091:         warnings.append("String type migration applied; bytes/text compatibility should be re
0092:
0093:     return {"success": True, "warnings": warnings}
0094:
0095: def _get_function_names(self, code: str) -> set:
0096:     try:
0097:         tree = ast.parse(code)

```

```
0098:         except SyntaxError:
0099:             return set()
0100:     return {node.name for node in ast.walk(tree) if isinstance(node, ast.FunctionDef)}
```

9. I. core/report_generator.py - Report output

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/report_generator.py

Simple explanation: this code block (lines 1 to 62) mainly handles human-readable report construction. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: human-readable report construction.
- Implementation density in this block: functions=1, classes=1, endpoint decorators=0.
- Key functions to study first: generate_report.
- Key classes in this block: ReportGenerator.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import datetime
0002: from typing import Dict, List, Optional
0003:
0004:
0005: class ReportGenerator:
0006:     def generate_report(
0007:         self,
0008:         analysis: dict,
0009:         selected_plans: List[dict],
0010:         success: bool,
0011:         validation_logs: list,
0012:         validation_result: Optional[Dict[str, str]] = None,
0013:     ) -> str:
0014:         timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
0015:         status = "SUCCESS" if success else "FAILED"
0016:
0017:         report = "# Legacy Python Modernization Report\n"
0018:         report += f"***Date:** {timestamp}\n"
0019:         report += f"***Status:** {status}\n"
0020:         report += f"***Probable Source Version:** {analysis.get('probable_source_version', 'Unknown')}\n"
0021:         report += f"***Execution Mode:** {analysis.get('mode', 'N/A')}\n\n"
0022:
0023:         report += "## Analysis Summary\n"
0024:         report += f"- **LOC:** {analysis.get('loc', 'N/A')}\n"
0025:         report += f"- **Complexity:** {analysis.get('complexity', 'N/A')}\n"
0026:         report += f"- **Risk Score:** {analysis.get('risk_score', 'N/A')}\n"
0027:         report += f"- **Legacy Issues:** {len(analysis.get('legacy_issues', []))}\n\n"
0028:
0029:         report += "## Planned Transformations\n"
```

```

0030:         if selected_plans:
0031:             for plan in selected_plans:
0032:                 suggestion = plan.get("suggestion", {})
0033:                 report += (
0034:                     f"- `{suggestion.get('id', 'unknown')}` on line {suggestion.get('line', 'N/A')}"
0035:                     f"{suggestion.get('reasoning', 'N/A')}" "
0036:                     f"(score={plan.get('safety_score', 'N/A')})\n"
0037:                 )
0038:             else:
0039:                 report += "- No transformations were selected.\n"
0040:             report += "\n"
0041:
0042:             report += "## Semantic Risks\n"
0043:             if analysis.get("semantic_risks"):
0044:                 for risk in analysis["semantic_risks"]:
0045:                     report += f"- {risk['message']} ({risk['risk']})\n"
0046:             else:
0047:                 report += "- No additional semantic risks were detected.\n"
0048:             report += "\n"
0049:
0050:             report += "## Validation\n"
0051:             if validation_result:
0052:                 report += f"- **Stage:** {validation_result.get('stage', 'N/A')}\n"
0053:                 report += f"- **Message:** {validation_result.get('message', validation_result.get('e"
0054:                 for warning in validation_result.get("warnings", []):
0055:                     report += f"- **Warning:** {warning}\n"
0056:                 report += "\n"
0057:
0058:             report += "## Execution Logs\n"
0059:             for log in validation_logs:
0060:                 report += f"- {log}\n"
0061:
0062:             return report

```


10. J. core/ml_reasoner.py - Optional model path

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/ml_reasoner.py

Simple explanation: this code block (lines 1 to 104) mainly handles optional model-assisted modernization path. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: optional model-assisted modernization path.
- Implementation density in this block: functions=5, classes=2, endpoint decorators=0.
- Key functions to study first: `__init__`, `is_available`, `explain_status`, `modernize`, `_ensure_loaded`.
- Key classes in this block: `MLInferenceResult`, `MLReasoner`.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import os
0002: from dataclasses import dataclass
0003: from typing import Optional
0004:
0005:
0006: PROMPT_TEMPLATE = """You are Helix AI, a legacy Python modernization model.
0007: Convert old Python code to current Python while preserving behavior.
0008: Return only the final migrated code.
0009:
0010: ### Instruction:
0011: Modernize this legacy Python code to current Python while preserving behavior.
0012:
0013: ### Input:
0014: {input_code}
0015:
0016: ### Response:
0017: """
0018:
0019:
0020: @dataclass
0021: class MLInferenceResult:
0022:     enabled: bool
0023:     available: bool
0024:     output: str = ""
0025:     error: str = ""
0026:     model_path: str = ""
0027:
0028:
0029: class MLReasoner:
0030:     def __init__(self, enabled: bool, adapter_path: str, base_model: str):
0031:         self.enabled = enabled
```

```

0032:         self.adapter_path = adapter_path
0033:         self.base_model = base_model
0034:         self._model = None
0035:         self._tokenizer = None
0036:         self._device = "cpu"
0037:         self._load_error: Optional[str] = None
0038:
0039:     def is_available(self) -> bool:
0040:         return self.enabled and bool(self.adapter_path) and os.path.exists(self.adapter_path)
0041:
0042:     def explain_status(self) -> dict:
0043:         return {
0044:             "enabled": self.enabled,
0045:             "available": self.is_available(),
0046:             "model_path": self.adapter_path,
0047:             "error": self._load_error,
0048:         }
0049:
0050:     def modernize(self, code: str, max_new_tokens: int = 256) -> MLInferenceResult:
0051:         if not self.enabled:
0052:             return MLInferenceResult(enabled=False, available=False, error="ML model is disabled.")
0053:         if not self.is_available():
0054:             return MLInferenceResult(
0055:                 enabled=True,
0056:                 available=False,
0057:                 error="Adapter path is missing or not trained yet.",
0058:                 model_path=self.adapter_path,
0059:             )
0060:
0061:         try:
0062:             self._ensure_loaded()
0063:             prompt = PROMPT_TEMPLATE.format(input_code=code)
0064:             inputs = self._tokenizer([prompt], return_tensors="pt").to(self._device)
0065:             outputs = self._model.generate(**inputs, max_new_tokens=max_new_tokens)
0066:             decoded = self._tokenizer.decode(outputs[0], skip_special_tokens=True)
0067:             final_output = decoded.split("### Response:")[1].strip()
0068:             return MLInferenceResult(
0069:                 enabled=True,
0070:                 available=True,
0071:                 output=final_output,
0072:                 model_path=self.adapter_path,
0073:             )
0074:         except Exception as exc:
0075:             self._load_error = str(exc)
0076:             return MLInferenceResult(
0077:                 enabled=True,
0078:                 available=False,
0079:                 error=str(exc),
0080:                 model_path=self.adapter_path,
0081:             )
0082:
0083:     def _ensure_loaded(self) -> None:
0084:         if self._model is not None and self._tokenizer is not None:
0085:             return
0086:
0087:         try:
0088:             import torch
0089:             from peft import PeftModel
0090:             from transformers import AutoModelForCausalLM, AutoTokenizer
0091:         except ImportError as exc:
0092:             raise RuntimeError("ML dependencies are missing. Install requirements-ml.txt.") from exc
0093:
0094:         self._device = "mps" if torch.backends.mps.is_available() else ("cuda" if torch.cuda.is_available() else "cpu")
0095:         try:
0096:             self._tokenizer = AutoTokenizer.from_pretrained(self.adapter_path)
0097:         except Exception:
0098:             self._tokenizer = AutoTokenizer.from_pretrained(self.base_model)
0099:         model = AutoModelForCausalLM.from_pretrained(
0100:             self.base_model,
0101:             torch_dtype=torch.float16 if self._device != "cpu" else torch.float32,

```

```
0102:         )
0103:         self._model = PeftModel.from_pretrained(model, self.adapter_path)
0104:         self._model.to(self._device)
```

11. K. static/js/app.js - UI state machine Part 1

File: /Users/rutwikbhondave/Desktop/Helix AI Project/static/js/app.js

Simple explanation: this code block (lines 1 to 260) mainly handles frontend interaction logic and UI state machine. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: frontend interaction logic and UI state machine.
- Implementation density in this block: functions=2, classes=0, endpoint decorators=0.
- Key functions to study first: emit_values, normalize_name.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: const SAMPLE_SNIPPETS = {
0002:     print: `for i in xrange(3):
0003:         print i`,
0004:     dict: `def emit_values(payload):
0005:         if payload.has_key("alpha"):
0006:             for key, value in payload.iteritems():
0007:                 print key, value`,
0008:     input: `def normalize_name(name):
0009:         if isinstance(name, basestring):
0010:             return unicode(name).strip()
0011:         return raw_input("fallback: ")`,
0012: };
0013:
0014: const state = {
0015:     originalCode: "",
0016:     newCode: "",
0017:     diff: "",
0018:     report: "",
0019:     runId: "",
0020:     analysis: null,
0021:     suggestions: [],
0022:     critiques: [],
0023:     plan: null,
0024:     validation: null,
0025:     ml: null,
0026:     logs: [],
0027: };
0028:
0029: const codeInput = document.getElementById("codeInput");
0030: const lineNumbers = document.getElementById("lineNumbers");
0031: const reviewPanel = document.getElementById("reviewPanel");
```

```

0032: const diffPanel = document.getElementById("diffPanel");
0033: const outputPanel = document.getElementById("outputPanel");
0034: const reportPanel = document.getElementById("reportPanel");
0035: const terminal = document.getElementById("terminal");
0036: const runHistory = document.getElementById("runHistory");
0037: const mlSummary = document.getElementById("mlSummary");
0038: const mlBadge = document.getElementById("mlBadge");
0039: const focusText = document.getElementById("focusText");
0040: const modeBadge = document.getElementById("modeBadge");
0041: const statusBadge = document.getElementById("statusBadge");
0042: const summaryStrip = document.getElementById("summaryStrip");
0043: const summarySource = document.getElementById("summarySource");
0044: const summaryIssues = document.getElementById("summaryIssues");
0045: const summaryChanges = document.getElementById("summaryChanges");
0046: const summaryState = document.getElementById("summaryState");
0047: const advancedPanel = document.getElementById("advancedPanel");
0048: const advancedToggleBtn = document.getElementById("advancedToggleBtn");
0049: const advancedToggleLabel = document.getElementById("advancedToggleLabel");
0050: const runBtn = document.getElementById("runBtn");
0051: const actionHint = document.getElementById("actionHint");
0052: const downloadReport = document.getElementById("downloadReport");
0053: const copyDiffBtn = document.getElementById("copyDiffBtn");
0054: const copyOutputBtn = document.getElementById("copyOutputBtn");
0055:
0056: const tabs = {
0057:   review: document.getElementById("reviewTab"),
0058:   diff: document.getElementById("diffTab"),
0059:   output: document.getElementById("outputTab"),
0060:   report: document.getElementById("reportTab"),
0061: };
0062:
0063: const panels = {
0064:   review: reviewPanel,
0065:   diff: diffPanel,
0066:   output: outputPanel,
0067:   report: reportPanel,
0068: };
0069:
0070: const steps = {
0071:   analyze: document.getElementById("stepAnalyze"),
0072:   review: document.getElementById("stepReview"),
0073:   modernize: document.getElementById("stepModernize"),
0074:   validate: document.getElementById("stepValidate"),
0075: };
0076:
0077: document.addEventListener("DOMContentLoaded", () => {
0078:   bindEvents();
0079:   resetUi();
0080:   fetchModelStatus();
0081:   loadRunHistory();
0082: });
0083:
0084: function bindEvents() {
0085:   codeInput.addEventListener("input", updateLineNumbers);
0086:   codeInput.addEventListener("scroll", syncScroll);
0087:
0088:   document.querySelectorAll(".sampleBtn").forEach((button) => {
0089:     button.addEventListener("click", () => {
0090:       const sample = SAMPLE_SNIPPETS[button.dataset.sample];
0091:       if (!sample) {
0092:         return;
0093:       }
0094:       codeInput.value = sample;
0095:       updateLineNumbers();
0096:       syncScroll();
0097:       showToast("Sample loaded", "success");
0098:     });
0099:   });
0100:
0101:   document.addEventListener("keydown", (event) => {

```

```

0102:         if ((event.metaKey || event.ctrlKey) && event.key === "Enter") {
0103:             event.preventDefault();
0104:             runBtn.click();
0105:         }
0106:     });
0107:
0108:     Object.entries(tabs).forEach(([key, button]) => {
0109:         button.addEventListener("click", () => activateTab(key));
0110:     });
0111:
0112:     Object.entries(steps).forEach(([key, button]) => {
0113:         button.addEventListener("click", () => handleStepNavigation(key));
0114:     });
0115:
0116:     advancedToggleBtn.addEventListener("click", () => setAdvancedOpen(advancedPanel.classList.contains("active")));
0117:     runHistory.addEventListener("click", handleRunHistoryClick);
0118:     runBtn.addEventListener("click", handlePrimaryAction);
0119:     copyDiffBtn.addEventListener("click", () => copyPanelContent("diff"));
0120:     copyOutputBtn.addEventListener("click", () => copyPanelContent("output"));
0121: }
0122:
0123: async function handlePrimaryAction() {
0124:     const action = state.primaryAction || "analyze";
0125:     if (action === "analyze") {
0126:         await handleAnalyze();
0127:         return;
0128:     }
0129:     if (action === "modernize") {
0130:         await handleModernize();
0131:         return;
0132:     }
0133:     if (action === "apply") {
0134:         applyOutputToEditor();
0135:     }
0136: }
0137:
0138: async function handleAnalyze() {
0139:     const code = codeInput.value;
0140:     if (!code.trim()) {
0141:         showToast("Paste legacy Python before analyzing", "error");
0142:         return;
0143:     }
0144:
0145:     state.originalCode = code;
0146:     state.newCode = "";
0147:     state.diff = "";
0148:     state.report = "";
0149:     state.runId = "";
0150:     state.validation = null;
0151:     state.logs = [];
0152:
0153:     resetUi(false);
0154:     setStepState("analyze");
0155:     setFocus("Analyzing your code to find legacy patterns and decide what can be modernized automatically");
0156:     setStatus("Analyzing");
0157:     setPrimaryAction("working");
0158:     log("Starting analysis...", "system");
0159:
0160:     try {
0161:         const response = await fetch("/analyze", {
0162:             method: "POST",
0163:             headers: { "Content-Type": "application/json" },
0164:             body: JSON.stringify({ code }),
0165:         });
0166:         const data = await response.json();
0167:
0168:         if (!response.ok || !data.analysis?.success) {
0169:             throw new Error(data.detail || data.analysis?.error || "Analysis failed.");
0170:         }
0171:     }

```

```

0172:     state.analysis = data.analysis;
0173:     state.suggestions = data.suggestions || [];
0174:     state.critiques = data.critiques?.critiques || [];
0175:     state.plan = data.plan || { selected_plans: [] };
0176:     state.ml = data.ml_reasoner || state.ml;
0177:
0178:     log(`Detected source profile: ${state.analysis.probable_source_version}`, "analyzer");
0179:     log(`Found ${state.analysis.legacy_issues?.length || 0} legacy issues.`, "analyzer");
0180:     log(`${(state.plan.selected_plans || []).length} modernization changes are ready to run.`);
0181:
0182:     renderSummary();
0183:     renderReview();
0184:     renderMlSummary(state.ml);
0185:     activateTab("review");
0186:
0187:     if (state.analysis.mode === "BLOCKED") {
0188:         setMode("Blocked");
0189:         setStatus("Blocked");
0190:         setStepState("review", ["analyze"]);
0191:         setFocus("Analysis found risky constructs that block automatic modernization. Review findings on the right.");
0192:         setPrimaryAction("blocked");
0193:         showToast("Automatic modernization is blocked for this code", "error");
0194:         return;
0195:     }
0196:
0197:     setMode(state.analysis.mode || "Safe");
0198:     setStatus((state.plan.selected_plans || []).length ? "Ready To Modernize" : "Review Findings");
0199:     setStepState("review", ["analyze"]);
0200:     setFocus(
0201:         (state.plan.selected_plans || []).length
0202:         ? "Review the findings on the right, then click Modernize Code to generate the updates."
0203:         : "Analysis is complete. Review the findings on the right. No automatic modernization is available."
0204:     );
0205:     setPrimaryAction((state.plan.selected_plans || []).length ? "modernize" : "analyze");
0206:     showToast("Analysis complete", "success");
0207: } catch (error) {
0208:     setMode("Error");
0209:     setStatus("Analysis Failed");
0210:     setFocus("Analysis failed. Check the advanced log and try again.");
0211:     setPrimaryAction("analyze");
0212:     log(error.message || "Analysis failed.", "error");
0213:     showToast("Analysis failed", "error");
0214: }
0215: }
0216:
0217: async function handleModernize() {
0218:     const code = codeInput.value;
0219:     state.originalCode = code;
0220:     setStepState("modernize", ["analyze", "review"]);
0221:     setStatus("Modernizing");
0222:     setFocus("Running the guarded modernization pipeline. The editor will stay unchanged until you click Stop Modernization.");
0223:     setPrimaryAction("working");
0224:     log("Running modernization pipeline...", "system");
0225:
0226:     try {
0227:         const response = await fetch("/refactor", {
0228:             method: "POST",
0229:             headers: { "Content-Type": "application/json" },
0230:             body: JSON.stringify({ code }),
0231:         });
0232:         const data = await response.json();
0233:
0234:         if (!response.ok || data.error) {
0235:             throw new Error(data.detail || data.error || "Modernization failed.");
0236:         }
0237:
0238:         state.runId = data.run_id || "";
0239:         state.analysis = data.analysis || state.analysis;
0240:         state.suggestions = data.suggestions || state.suggestions;
0241:         state.plan = data.plan || state.plan;

```

```
0242:     state.validation = data.validation || null;
0243:     state.newCode = data.new_code || "";
0244:     state.diff = data.diff || "";
0245:     state.report = data.report || "";
0246:     state.ml = data.ml_reasoner || state.ml;
0247:     state.logs = data.logs || [];
0248:
0249:     renderSummary();
0250:     renderReview();
0251:     renderDiff();
0252:     renderOutput();
0253:     renderReport();
0254:     renderLogs();
0255:     renderMlSummary(state.ml);
0256:     activateTab(state.diff ? "diff" : "output");
0257:
0258:     if (state.report) {
0259:         const blob = new Blob([state.report], { type: "text/markdown" });
0260:         downloadReport.href = URL.createObjectURL(blob);
```


12. L. static/js/app.js - UI state machine Part 2

File: /Users/rutwikbhondave/Desktop/Helix AI Project/static/js/app.js

Simple explanation: this code block (lines 261 to 520) mainly handles frontend interaction logic and UI state machine. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: frontend interaction logic and UI state machine.
- Implementation density in this block: functions=0, classes=0, endpoint decorators=0.
- Validation-related checks are present, so this block contributes to safety gating.
- Risk/warning language is present, indicating this block is part of decision transparency.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0261:         downloadReport.classList.remove("hidden");
0262:     }
0263:
0264:     setStepState("validate", ["analyze", "review", "modernize"]);
0265:     setMode(state.analysis?.mode || "Safe");
0266:     setStatus(state.validation?.success ? "Validated" : "Needs Review");
0267:     setFocus(
0268:         state.validation?.success
0269:         ? "Modernization finished. Review the diff, then apply the updated code if you want to proceed."
0270:         : "Modernization finished with warnings. Review the diff and report before trusting the output."
0271:     );
0272:     setPrimaryAction(state.newCode && state.newCode !== state.originalCode ? "apply" : "analyze");
0273:
0274:     if (state.runId) {
0275:         log(`Saved run ${state.runId}.`, "success");
0276:         loadRunHistory();
0277:     }
0278:
0279:     showToast(state.validation?.success ? "Modernization completed" : "Modernization finished with warnings", "info");
0280: } catch (error) {
0281:     setMode("Error");
0282:     setStatus("Modernization Failed");
0283:     setFocus("Modernization failed. Check the advanced log and try again.");
0284:     setPrimaryAction("modernize");
0285:     log(error.message || "Modernization failed.", "error");
0286:     showToast("Modernization failed", "error");
0287: }
0288: }
0289:
0290: function applyOutputToEditor() {
0291:     if (!state.newCode) {
0292:         return;
0293:     }
```

```

0294:     codeInput.value = state.newCode;
0295:     updateLineNumbers();
0296:     syncScroll();
0297:     setPrimaryAction("analyze");
0298:     setFocus("Changes were applied to the editor. You can analyze again or continue editing.");
0299:     showToast("Updated code applied to editor", "success");
0300: }
0301:
0302: function handleStepNavigation(step) {
0303:     if (step === "analyze") {
0304:         codeInput.focus();
0305:         codeInput.scrollIntoView({ behavior: "smooth", block: "center" });
0306:         showToast("Editor focused", "info");
0307:         return;
0308:     }
0309:
0310:     if (step === "review") {
0311:         if (!state.analysis) {
0312:             showToast("Analyze the code first", "error");
0313:             return;
0314:         }
0315:         activateTab("review");
0316:         showToast("Review opened", "success");
0317:         return;
0318:     }
0319:
0320:     if (step === "modernize") {
0321:         if (!state.analysis) {
0322:             showToast("Analyze the code first", "error");
0323:             return;
0324:         }
0325:         activateTab("review");
0326:         runBtn.scrollIntoView({ behavior: "smooth", block: "center" });
0327:         runBtn.focus();
0328:         showToast(state.primaryAction === "modernize" ? "Ready to modernize" : "Review the findings");
0329:         return;
0330:     }
0331:
0332:     if (step === "validate") {
0333:         if (!state.diff && !state.report && !state.newCode) {
0334:             showToast("Run modernization to review the result", "error");
0335:             return;
0336:         }
0337:         const targetTab = state.diff ? "diff" : (state.newCode ? "output" : "report");
0338:         activateTab(targetTab);
0339:         showToast("Validation view opened", "success");
0340:     }
0341: }
0342:
0343: async function fetchModelStatus() {
0344:     try {
0345:         const response = await fetch("/model-status");
0346:         const data = await response.json();
0347:         state.ml = data;
0348:         renderMlSummary(data);
0349:     } catch (error) {
0350:         renderMlSummary(null);
0351:     }
0352: }
0353:
0354: async function loadRunHistory() {
0355:     try {
0356:         const response = await fetch("/runs?limit=6");
0357:         const data = await response.json();
0358:         const runs = data.runs || [];
0359:
0360:         if (!runs.length) {
0361:             runHistory.innerHTML = `<div class="rounded-xl border border-dashed border-white/10 bg-white/10 p-4">
0362:             return;
0363:         }

```

```

0364:
0365:     runHistory.innerHTML = runs.map((run) => `
0366:         <button data-run-id="${escapeHtml(run.run_id)}" class="w-full rounded-xl border border
0367:             <div class="flex items-center justify-between gap-3">
0368:                 <div class="font-mono text-[11px] text-zinc-300">${escapeHtml(run.run_id.slice
0369:                 <span class="text-[10px] uppercase tracking-[0.18em] ${run.validation_success
0370:             </div>
0371:             <div class="mt-2 text-[11px] text-zinc-400">${escapeHtml(run.probable_source_vers
0372:         </button>
0373:     `).join("");
0374: } catch (error) {
0375:     runHistory.innerHTML = `<div class="rounded-xl border border-rose-500/20 bg-rose-500/10 p
0376: }
0377: }
0378:
0379: async function handleRunHistoryClick(event) {
0380:     const target = event.target.closest("[data-run-id]");
0381:     if (!target) {
0382:         return;
0383:     }
0384:
0385:     try {
0386:         const response = await fetch(`/runs/${target.dataset.runId}`);
0387:         const data = await response.json();
0388:         if (!response.ok) {
0389:             throw new Error(data.detail || "Run not found.");
0390:         }
0391:
0392:         state.runId = data.run_id || "";
0393:         state.originalCode = data.original_code || "";
0394:         state.newCode = data.new_code || "";
0395:         state.analysis = data.analysis || null;
0396:         state.validation = data.validation || null;
0397:         state.diff = data.diff || "";
0398:         state.report = data.report || "";
0399:         state.logs = [
0400:             `Loaded saved run ${state.runId}.`,
0401:             `Validation status: ${data.validation_success ? "validated" : "needs review"}.`,
0402:         ];
0403:         state.plan = {
0404:             selected_plans: (data.applied_transformations || []).map((id) => ({ suggestion: { id
0405:         });
0406:
0407:         renderSummary();
0408:         renderReview();
0409:         renderDiff();
0410:         renderOutput();
0411:         renderReport();
0412:         renderLogs();
0413:         setAdvancedOpen(true);
0414:         activateTab(state.diff ? "diff" : "report");
0415:         setStepState("validate", ["analyze", "review", "modernize"]);
0416:         setMode(state.analysis?.mode || "Review");
0417:         setStatus(data.validation_success ? "Validated" : "Needs Review");
0418:         setFocus("Saved run loaded. Review the diff, output, or report without changing the editor
0419:         setPrimaryAction(state.newCode && state.newCode !== state.originalCode ? "apply" : "analy
0420:         showToast("Saved run loaded", "success");
0421:     } catch (error) {
0422:         showToast(error.message || "Failed to load saved run", "error");
0423:     }
0424: }
0425:
0426: function renderSummary() {
0427:     if (!state.analysis) {
0428:         summaryStrip.classList.add("hidden");
0429:         return;
0430:     }
0431:
0432:     summaryStrip.classList.remove("hidden");
0433:     summarySource.textContent = state.analysis.probable_source_version || "Unknown";

```

```

0434:     summaryIssues.textContent = String(state.analysis.legacy_issues?.length || 0);
0435:     summaryChanges.textContent = String(state.plan?.selected_plans?.length || 0);
0436:     summaryState.textContent = state.validation
0437:       ? (state.validation.success ? "Validated" : "Review")
0438:       : ((state.plan?.selected_plans?.length || 0) ? "Ready" : "Waiting");
0439:   }
0440:
0441:   function renderReview() {
0442:     if (!state.analysis) {
0443:       renderPanelPlaceholder(
0444:         reviewPanel,
0445:         "Analysis results will appear here",
0446:         "Start with Analyze Code to detect legacy syntax, compatibility risks, and available m
0447:         "Load a sample or paste Python 2 style code into the editor."
0448:       );
0449:       return;
0450:     }
0451:
0452:     const issues = state.analysis.legacy_issues || [];
0453:     const plans = state.plan?.selected_plans || [];
0454:     const warnings = [
0455:       ...(state.analysis.semantic_risks || []).map((risk) => risk.message),
0456:       ...(state.validation?.warnings || []),
0457:     ];
0458:     const riskTone = riskToneClasses(state.analysis.risk_score);
0459:
0460:     reviewPanel.innerHTML = `
0461:       <div class="panel-shell space-y-4">
0462:         <div class="rounded-2xl border border-white/10 bg-black/20 p-5">
0463:           <div class="flex flex-col gap-4 md:flex-row md:items-start md:justify-between">
0464:             <div>
0465:               <div class="text-lg font-semibold text-white">${escapeHtml(state.analysis
0466:               <div class="mt-2 text-sm text-zinc-400">${issues.length} legacy issue${iss
0467:             </div>
0468:             <span class="inline-flex rounded-full border px-3 py-1 text-[10px] font-semibol
0469:           </div>
0470:         </div>
0471:
0472:         <div class="grid gap-4 lg:grid-cols-2">
0473:           <div class="rounded-2xl border border-white/10 bg-black/20 p-4">
0474:             <div class="text-[11px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0475:               <div class="mt-3 space-y-2">
0476:                 ${issues.length}
0477:                 ? issues.slice(0, 5).map((issue) => `
0478:                   <div class="rounded-xl border border-white/5 bg-black/30 px-3 py-3"
0479:                     <div class="text-sm text-zinc-200">${escapeHtml(issue.message)}
0480:                     <div class="mt-1 text-[11px] text-zinc-500">Line ${issue.line}
0481:                   </div>
0482:                 `).join("")
0483:                 : `<div class="text-sm text-zinc-500">No legacy issues were detected.
0484:               </div>
0485:             </div>
0486:
0487:           <div class="rounded-2xl border border-white/10 bg-black/20 p-4">
0488:             <div class="text-[11px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0489:               <div class="mt-3 space-y-2">
0490:                 ${plans.length}
0491:                 ? plans.map((item, index) => `
0492:                   <div class="rounded-xl border border-white/5 bg-black/30 px-3 py-3"
0493:                     <div class="text-sm text-zinc-200">${index + 1}. ${escapeHtml
0494:                   </div>
0495:                 `).join("")
0496:                 : `<div class="text-sm text-zinc-500">No automatic changes are ready.
0497:               </div>
0498:             </div>
0499:           </div>
0500:
0501:           ${warnings.length ? `
0502:             <div class="rounded-2xl border border-amber-500/20 bg-amber-500/5 p-4">
0503:               <div class="text-[11px] font-mono uppercase tracking-[0.18em] text-amber-300">

```

```

0504:             <div class="mt-3 space-y-2">
0505:                 ${warnings.slice(0, 4).map((warning) => `<div class="text-sm text-amber-100">
0506:                     <div>
0507:                         </div>
0508:                     ` : ""}
0509:                 </div>
0510:             `;
0511: }
0512:
0513: function renderDiff() {
0514:     if (!state.diff) {
0515:         renderPanelPlaceholder(
0516:             diffPanel,
0517:             "Diff preview will appear here",
0518:             "After modernization, this panel will show every added and removed line before anything",
0519:             "Use the diff as the final review checkpoint."
0520:         );

```

13. M. templates/index.html - Inner UI structure

File: /Users/rutwikbhondave/Desktop/Helix AI Project/templates/index.html

Simple explanation: this code block (lines 1 to 280) mainly handles UI structure and layout for user workflow. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: UI structure and layout for user workflow.
- Implementation density in this block: functions=1, classes=0, endpoint decorators=0.
- Key functions to study first: `legacy_scan`.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: <!DOCTYPE html>
0002: <html lang="en" class="dark">
0003: <head>
0004:     <meta charset="UTF-8">
0005:     <meta name="viewport" content="width=device-width, initial-scale=1.0">
0006:     <title>Helix AI | Legacy Python Modernization</title>
0007:     <script src="https://cdn.tailwindcss.com"></script>
0008:     <link href="https://fonts.googleapis.com/css2?family=Geist:wght@300;400;500;600;700&display=swap" rel="stylesheet">
0009:     <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
0010:     <script>
0011:         tailwind.config = {
0012:             darkMode: "class",
0013:             theme: {
0014:                 extend: {
0015:                     fontFamily: {
0016:                         sans: ["Geist", "sans-serif"],
0017:                         mono: ["JetBrains Mono", "monospace"],
0018:                     },
0019:                 },
0020:             },
0021:         };
0022:     </script>
0023:     <style>
0024:         body { font-family: "Geist", sans-serif; }
0025:         @keyframes fadeInUp {
0026:             from { opacity: 0; transform: translateY(12px); }
0027:             to { opacity: 1; transform: translateY(0); }
0028:         }
0029:         @keyframes softPulse {
0030:             0%, 100% { box-shadow: 0 0 0 rgba(249, 115, 22, 0); }
0031:             50% { box-shadow: 0 0 24px rgba(249, 115, 22, 0.16); }
0032:         }
0033:         .animate-fade-in-up { animation: fadeInUp 0.25s ease-out forwards; }
```

```

0034: .custom-scrollbar::-webkit-scrollbar { width: 6px; height: 6px; }
0035: .custom-scrollbar::-webkit-scrollbar-track { background: #18181b; }
0036: .custom-scrollbar::-webkit-scrollbar-thumb { background: #3f3f46; border-radius: 999px; }
0037: .step-item,
0038: .sampleBtn,
0039: .tab-btn,
0040: #downloadReport,
0041: .copy-btn,
0042: #advancedToggleBtn,
0043: #summaryStrip > div,
0044: #runHistory button,
0045: header a[href="/"],
0046: .feature-pill {
0047:     transition: transform 180ms ease, border-color 180ms ease, background-color 180ms ease;
0048: }
0049: .feature-pill:hover {
0050:     transform: translateY(-1px);
0051:     color: #fff7ed;
0052:     border-color: rgba(249, 115, 22, 0.4);
0053:     background: rgba(249, 115, 22, 0.12);
0054:     box-shadow: 0 10px 22px rgba(249, 115, 22, 0.12);
0055: }
0056: .step-item:hover {
0057:     transform: translateY(-2px);
0058:     border-color: rgba(255, 255, 255, 0.10);
0059:     box-shadow: 0 14px 28px rgba(0, 0, 0, 0.18);
0060: }
0061: .step-item.current .step-dot {
0062:     border-color: rgba(249, 115, 22, 0.7);
0063:     background: rgba(249, 115, 22, 0.16);
0064:     color: #fff7ed;
0065:     box-shadow: 0 0 0 1px rgba(249, 115, 22, 0.16), 0 0 24px rgba(249, 115, 22, 0.12);
0066:     animation: softPulse 2.4s ease-in-out infinite;
0067: }
0068: .step-item.done .step-dot {
0069:     border-color: rgba(16, 185, 129, 0.6);
0070:     background: rgba(16, 185, 129, 0.14);
0071:     color: #d1fae5;
0072: }
0073: .step-item.done .step-label { color: #e5e7eb; }
0074: .step-item.current .step-label { color: #ffffff; }
0075: .step-item.current .step-copy { color: #a1a1aa; }
0076: #runBtn {
0077:     transition: transform 180ms ease, box-shadow 220ms ease, background-color 180ms ease;
0078: }
0079: #runBtn:not(:disabled):hover {
0080:     transform: translateY(-1px);
0081:     box-shadow: 0 14px 28px rgba(255, 255, 255, 0.10), 0 0 18px rgba(249, 115, 22, 0.10);
0082: }
0083: #runBtn:not(:disabled):active {
0084:     transform: translateY(0);
0085:     box-shadow: 0 6px 14px rgba(0, 0, 0, 0.20);
0086: }
0087: .sampleBtn:hover,
0088: .copy-btn:hover,
0089: #downloadReport:hover,
0090: header a[href="/"]:hover {
0091:     transform: translateY(-1px);
0092:     box-shadow: 0 10px 24px rgba(0, 0, 0, 0.18);
0093: }
0094: .tab-btn.active {
0095:     color: white;
0096:     border-bottom: 2px solid rgb(249 115 22);
0097:     background: rgba(255,255,255,0.05);
0098: }
0099: .tab-btn:hover {
0100:     color: #ffffff;
0101:     background: rgba(255,255,255,0.04);
0102: }
0103: #summaryStrip > div:hover,

```

```

0104:         #runHistory button:hover {
0105:             transform: translateY(-2px);
0106:             border-color: rgba(249, 115, 22, 0.22);
0107:             box-shadow: 0 14px 28px rgba(0, 0, 0, 0.18);
0108:         }
0109:         #advancedToggleBtn:hover {
0110:             background: rgba(255, 255, 255, 0.03);
0111:         }
0112:         #advancedToggleBtn:hover #advancedToggleLabel {
0113:             color: #f4f4f5;
0114:         }
0115:         .diff-line.add { color: #86efac; background: rgba(34, 197, 94, 0.08); }
0116:         .diff-line.remove { color: #fda4af; background: rgba(244, 63, 94, 0.08); }
0117:         .diff-line.meta { color: #dd3fc; }
0118:         .panel-shell {
0119:             animation: fadeInUp 0.22s ease-out;
0120:         }
0121:         .step-item button {
0122:             cursor: pointer;
0123:         }
0124:         @media (prefers-reduced-motion: reduce) {
0125:             .animate-fade-in-up,
0126:             .step-item.current .step-dot {
0127:                 animation: none !important;
0128:             }
0129:             .step-item,
0130:             .sampleBtn,
0131:             .tab-btn,
0132:             #runBtn,
0133:             #downloadReport,
0134:             .copy-btn,
0135:             #advancedToggleBtn,
0136:             #summaryStrip > div,
0137:             #runHistory button,
0138:             header a[href="/"],
0139:             .feature-pill {
0140:                 transition: none !important;
0141:             }
0142:         }
0143:     </style>
0144: </head>
0145: <body class="min-h-screen bg-black text-white antialiased selection:bg-orange-500/30 selection:text-white">
0146:     <div class="fixed inset-0 -z-10 pointer-events-none">
0147:         <div class="absolute inset-0 bg-[radial-gradient(circle_at_top,rgba(251,146,60,0.13),transparent_0%)]">
0148:         <div class="absolute inset-0 bg-[radial-gradient(circle_at_bottom_right,rgba(59,130,246,0.15),transparent_0%)]">
0149:         <div class="absolute inset-0 bg-[linear-gradient(rgba(255,255,255,0.03)_1px,transparent_1px)]">
0150:     </div>
0151:
0152:     <div class="mx-auto flex min-h-screen w-full max-w-7xl flex-col px-4 py-4 sm:px-6 lg:px-8">
0153:         <header class="rounded-3xl border border-white/10 bg-black/60 px-5 py-4 backdrop-blur-xl">
0154:             <div class="flex flex-col gap-4 lg:flex-row lg:items-center lg:justify-between">
0155:                 <div>
0156:                     <a href="/" class="inline-flex items-center gap-3 text-sm tracking-tight text-white">
0157:                         <span class="h-2 w-2 rounded-full bg-orange-500 shadow-[0_0_12px_rgba(249,115,22,0.22)]"></span>
0158:                         <span>Helix AI <span class="text-zinc-500"></span> <span class="text-zinc-500"></span>
0159:                     </a>
0160:                     <h1 class="mt-3 text-2xl font-semibold tracking-tight text-white">Modernize L
0161:                     <div class="mt-3 flex flex-wrap gap-2">
0162:                         <span class="feature-pill rounded-full border border-white/10 bg-zinc-900/70 px-2 py-1">
0163:                         <span class="feature-pill rounded-full border border-white/10 bg-zinc-900/70 px-2 py-1">
0164:                         <span class="feature-pill rounded-full border border-white/10 bg-zinc-900/70 px-2 py-1">
0165:                     </div>
0166:                 </div>
0167:                 <div class="flex items-center gap-3">
0168:                     <span id="modeBadge" class="rounded-full border border-white/10 bg-zinc-900/70 px-2 py-1">
0169:                     <a href="/" class="rounded-full border border-white/10 px-3 py-2 text-xs upper
0170:                 </div>
0171:             </div>
0172:         </header>
0173:

```



```

0174:         <section class="mt-5 rounded-3xl border border-white/10 bg-zinc-950/40 px-5 py-5">
0175:             <div class="grid gap-4 md:grid-cols-4">
0176:                 <button id="stepAnalyze" data-step-target="analyze" class="step-item current rounded-2xl border border-white/10 bg-zinc-900/80">
0177:                     <div class="step-dot flex h-10 w-10 items-center justify-center rounded-xl border border-white/10 bg-zinc-900/80">
0178:                         <svg class="h-4 w-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
0179:                             </div>
0180:                     <div class="step-label mt-3 text-sm font-medium text-zinc-500">Analyze</div>
0181:                     <div class="step-copy mt-1 text-xs text-zinc-600">Detect legacy syntax and con
0182:                 </button>
0183:                 <button id="stepReview" data-step-target="review" class="step-item rounded-2xl border border-white/10 bg-zinc-900/80">
0184:                     <div class="step-dot flex h-10 w-10 items-center justify-center rounded-xl border border-white/10 bg-zinc-900/80">
0185:                         <svg class="h-4 w-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
0186:                             </div>
0187:                     <div class="step-label mt-3 text-sm font-medium text-zinc-500">Review</div>
0188:                     <div class="step-copy mt-1 text-xs text-zinc-600">See what will change and wha
0189:                 </button>
0190:                 <button id="stepModernize" data-step-target="modernize" class="step-item rounded-2xl border border-white/10 bg-zinc-900/80">
0191:                     <div class="step-dot flex h-10 w-10 items-center justify-center rounded-xl border border-white/10 bg-zinc-900/80">
0192:                         <svg class="h-4 w-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
0193:                             </div>
0194:                     <div class="step-label mt-3 text-sm font-medium text-zinc-500">Modernize</div>
0195:                     <div class="step-copy mt-1 text-xs text-zinc-600">Run the guarded modernizati
0196:                 </button>
0197:                 <button id="stepValidate" data-step-target="validate" class="step-item rounded-2xl border border-white/10 bg-zinc-900/80">
0198:                     <div class="step-dot flex h-10 w-10 items-center justify-center rounded-xl border border-white/10 bg-zinc-900/80">
0199:                         <svg class="h-4 w-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
0200:                             </div>
0201:                     <div class="step-label mt-3 text-sm font-medium text-zinc-500">Validate</div>
0202:                     <div class="step-copy mt-1 text-xs text-zinc-600">Inspect the diff, output, an
0203:                 </button>
0204:             </div>
0205:         </section>
0206:
0207:         <main class="mt-5 grid flex-1 gap-5 lg:grid-cols-[1.08fr_0.92fr]">
0208:             <section class="flex min-h-[720px] flex-col rounded-3xl border border-white/10 bg-zinc-950/40">
0209:                 <div class="border-b border-white/10 px-5 py-4">
0210:                     <div class="flex flex-col gap-4 lg:flex-row lg:items-center lg:justify-between">
0211:                         <div>
0212:                             <div class="text-[11px] font-mono uppercase tracking-[0.2em] text-zinc-500">
0213:                                 <h2 class="mt-2 text-lg font-semibold text-white">Paste your legacy Py
0214:                             </div>
0215:                             <div class="flex flex-wrap items-center gap-3">
0216:                                 <button id="runBtn" class="rounded-full bg-white px-4 py-2 text-xs font-medium text-zinc-900">Run
0217:                                 <span id="actionHint" class="text-[10px] font-mono uppercase tracking-[0.2em] text-zinc-500">
0218:                             </div>
0219:                         </div>
0220:                         <div class="mt-4 flex flex-wrap gap-2">
0221:                             <button class="sampleBtn rounded-full border border-white/10 bg-zinc-900/80 px-4 py-2 text-xs font-medium text-zinc-900">Sample
0222:                             <button class="sampleBtn rounded-full border border-white/10 bg-zinc-900/80 px-4 py-2 text-xs font-medium text-zinc-900">Sample
0223:                             <button class="sampleBtn rounded-full border border-white/10 bg-zinc-900/80 px-4 py-2 text-xs font-medium text-zinc-900">Sample
0224:                         </div>
0225:                         <div class="mt-4 text-[10px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0226:                     </div>
0227:
0228:                     <div class="relative flex-1">
0229:                         <div id="lineNumbers" class="absolute bottom-0 left-0 top-0 w-12 select-none text-zinc-500">
0230:                             <textarea id="codeInput" class="absolute inset-0 resize-none bg-transparent p-4">
0231: print "starting migration"
0232: for i in xrange(len(values)):
0233:     if values.has_key(i):
0234:         print values[i]
0235: return raw_input("done? ")</textarea>
0236:                         </div>
0237:                     </section>
0238:
0239:             <section class="flex min-h-[720px] flex-col rounded-3xl border border-white/10 bg-black">
0240:                 <div class="border-b border-white/10 px-5 py-4">
0241:                     <div class="flex items-start justify-between gap-4">
0242:                         <div>
0243:                             <div class="text-[11px] font-mono uppercase tracking-[0.2em] text-zinc-500">

```

```

0244:         <h2 class="mt-2 text-lg font-semibold text-white">Review and modernize
0245:     </div>
0246:     <span id="statusBadge" class="rounded-full border border-white/10 bg-zinc-
0247: </div>
0248: <div class="mt-4 rounded-2xl border border-white/5 bg-zinc-950/60 p-4">
0249:     <div class="text-[10px] font-mono uppercase tracking-[0.2em] text-zinc-500">
0250:     <p id="focusText" class="mt-2 text-sm leading-6 text-zinc-300">Paste legal
0251: </div>
0252: <div id="summaryStrip" class="mt-4 hidden grid grid-cols-2 gap-2 md:grid-cols-2">
0253:     <div class="rounded-xl border border-white/10 bg-black/20 p-3">
0254:         <div class="text-[10px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0255:         <div id="summarySource" class="mt-2 text-sm text-white">Pending</div>
0256:     </div>
0257:     <div class="rounded-xl border border-white/10 bg-black/20 p-3">
0258:         <div class="text-[10px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0259:         <div id="summaryIssues" class="mt-2 text-sm text-white">0</div>
0260:     </div>
0261:     <div class="rounded-xl border border-white/10 bg-black/20 p-3">
0262:         <div class="text-[10px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0263:         <div id="summaryChanges" class="mt-2 text-sm text-white">0</div>
0264:     </div>
0265:     <div class="rounded-xl border border-white/10 bg-black/20 p-3">
0266:         <div class="text-[10px] font-mono uppercase tracking-[0.18em] text-zinc-500">
0267:         <div id="summaryState" class="mt-2 text-sm text-white">Waiting</div>
0268:     </div>
0269: </div>
0270: </div>
0271:
0272: <div class="border-b border-white/10 px-5">
0273:     <div class="flex flex-wrap gap-1">
0274:         <button id="reviewTab" class="tab-btn active px-4 py-3 text-sm font-medium text-white">
0275:         <button id="diffTab" class="tab-btn px-4 py-3 text-sm font-medium text-zinc-500">
0276:         <button id="outputTab" class="tab-btn px-4 py-3 text-sm font-medium text-zinc-500">
0277:         <button id="reportTab" class="tab-btn px-4 py-3 text-sm font-medium text-zinc-500">
0278:     </div>
0279: </div>
0280:

```

14. N. ml/train_modernizer_unsloth.py - Training core

File: /Users/rutwikbhondave/Desktop/Helix AI Project/ml/train_modernizer_unsloth.py

Simple explanation: this code block (lines 1 to 135) mainly handles model fine-tuning workflow and optimization configuration. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: model fine-tuning workflow and optimization configuration.
- Implementation density in this block: functions=3, classes=0, endpoint decorators=0.
- Key functions to study first: load_records, formatting_prompts_func, main.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import argparse
0002: import json
0003: from pathlib import Path
0004:
0005:
0006: DEFAULT_BASE_MODEL = "unsloth/Qwen2.5-Coder-1.5B-Instruct"
0007: DEFAULT_DATA_PATH = Path("ml/data/modernization_train.jsonl")
0008: DEFAULT_OUTPUT_DIR = Path("ml/models/nebula-modernizer-qwen25-1.5b")
0009: MAX_SEQ_LENGTH = 2048
0010:
0011:
0012: PROMPT_TEMPLATE = """You are Helix AI, a legacy Python modernization model.
0013: Convert old Python code to current Python while preserving behavior.
0014: Return only the final migrated code.
0015:
0016: """ Instruction:
0017: {instruction}
0018:
0019: """ Input:
0020: {input_code}
0021:
0022: """ Response:
0023: {output_code}"""
0024:
0025:
0026: def load_records() -> list[dict]:
0027:     records = []
0028:     with DATA_PATH.open(encoding="utf-8") as handle:
0029:         for line in handle:
0030:             line = line.strip()
0031:             if line:
0032:                 records.append(json.loads(line))
0033:     return records
```

```

0034:
0035:
0036: def formatting_prompts_func(examples):
0037:     texts = []
0038:     for instruction, input_code, output_code in zip(
0039:         examples["instruction"],
0040:         examples["input"],
0041:         examples["output"],
0042:     ):
0043:         texts.append(
0044:             PROMPT_TEMPLATE.format(
0045:                 instruction=instruction,
0046:                 input_code=input_code,
0047:                 output_code=output_code,
0048:             )
0049:         )
0050:     return {"text": texts}
0051:
0052:
0053: def main() -> None:
0054:     try:
0055:         from datasets import Dataset
0056:         from trl import SFTConfig, SFTTrainer
0057:         from unsloth import FastLanguageModel
0058:     except ImportError as exc:
0059:         raise SystemExit(
0060:             "Missing ML dependencies. Install with `pip install -r requirements-ml.txt`."
0061:         ) from exc
0062:
0063:     parser = argparse.ArgumentParser()
0064:     parser.add_argument("--base-model", default=DEFAULT_BASE_MODEL)
0065:     parser.add_argument("--data-path", default=str(DEFAULT_DATA_PATH))
0066:     parser.add_argument("--output-dir", default=str(DEFAULT_OUTPUT_DIR))
0067:     parser.add_argument("--batch-size", type=int, default=2)
0068:     parser.add_argument("--gradient-accumulation-steps", type=int, default=4)
0069:     parser.add_argument("--epochs", type=int, default=3)
0070:     parser.add_argument("--learning-rate", type=float, default=2e-4)
0071:     args = parser.parse_args()
0072:
0073:     data_path = Path(args.data_path)
0074:     output_dir = Path(args.output_dir)
0075:
0076:     if not data_path.exists():
0077:         raise SystemExit(
0078:             f"Training data not found at {data_path}. Run `python3 ml/build_seed_modernization_data.py`"
0079:             f"and `python3 ml/curate_dataset.py` first."
0080:         )
0081:
0082:     global DATA_PATH
0083:     DATA_PATH = data_path
0084:     raw_records = load_records()
0085:     dataset = Dataset.from_list(raw_records)
0086:     dataset = dataset.shuffle(seed=42)
0087:     dataset = dataset.map(formatting_prompts_func, batched=True)
0088:
0089:     model, tokenizer = FastLanguageModel.from_pretrained(
0090:         model_name=args.base_model,
0091:         max_seq_length=MAX_SEQ_LENGTH,
0092:         dtype=None,
0093:         load_in_4bit=True,
0094:     )
0095:
0096:     model = FastLanguageModel.get_peft_model(
0097:         model,
0098:         r=16,
0099:         target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
0100:         lora_alpha=16,
0101:         lora_dropout=0,
0102:         bias="none",
0103:         use_gradient_checkpointing="unsloth",

```

```

0104:         random_state=42,
0105:     )
0106:
0107:     trainer = SFTTrainer(
0108:         model=model,
0109:         tokenizer=tokenizer,
0110:         train_dataset=dataset,
0111:         dataset_text_field="text",
0112:         max_seq_length=MAX_SEQ_LENGTH,
0113:         args=SFTConfig(
0114:             output_dir=str(output_dir),
0115:             per_device_train_batch_size=args.batch_size,
0116:             gradient_accumulation_steps=args.gradient_accumulation_steps,
0117:             learning_rate=args.learning_rate,
0118:             num_train_epochs=args.epochs,
0119:             warmup_ratio=0.05,
0120:             logging_steps=5,
0121:             save_strategy="epoch",
0122:             lr_scheduler_type="cosine",
0123:             weight_decay=0.01,
0124:             report_to=[],
0125:         ),
0126:     )
0127:
0128:     trainer.train()
0129:     model.save_pretrained(str(output_dir))
0130:     tokenizer.save_pretrained(str(output_dir))
0131:     print(f"Saved adapter to {output_dir}")
0132:
0133:
0134: if __name__ == "__main__":
0135:     main()

```

15. O. ml/build_training_mixture.py - Mixture builder

File: /Users/rutwikbhondave/Desktop/Helix AI Project/ml/build_training_mixture.py

Simple explanation: this code block (lines 1 to 52) mainly handles model fine-tuning workflow and optimization configuration. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: model fine-tuning workflow and optimization configuration.
- Implementation density in this block: functions=2, classes=0, endpoint decorators=0.
- Key functions to study first: load_jsonl, main.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import json
0002: from pathlib import Path
0003:
0004:
0005: MANIFEST_PATH = Path("ml/public_dataset_manifest.json")
0006: OUTPUT_PATH = Path("ml/data/training_mixture.jsonl")
0007:
0008:
0009: def load_jsonl(path: Path) -> list[dict]:
0010:     records = []
0011:     if not path.exists():
0012:         return records
0013:     with path.open(encoding="utf-8") as handle:
0014:         for line in handle:
0015:             line = line.strip()
0016:             if line:
0017:                 records.append(json.loads(line))
0018:     return records
0019:
0020:
0021: def main() -> None:
0022:     manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
0023:     OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)
0024:
0025:     mixture = []
0026:     for source in manifest["sources"]:
0027:         path_value = source.get("path") or source.get("prepared_path")
0028:         if not path_value:
0029:             continue
0030:
0031:         path = Path(path_value)
0032:         if not path.exists():
0033:             print(f"Skipping missing source: {path}")
0034:             continue
0035:
```

```

0036:         records = load_jsonl(path)
0037:         weight = int(source.get("weight", 1))
0038:         for record in records:
0039:             enriched = dict(record)
0040:             enriched["mixture_source"] = source["name"]
0041:             for _ in range(max(weight, 1)):
0042:                 mixture.append(enriched)
0043:
0044:         with OUTPUT_PATH.open("w", encoding="utf-8") as handle:
0045:             for record in mixture:
0046:                 handle.write(json.dumps(record, ensure_ascii=True) + "\n")
0047:
0048:         print(f"Wrote {len(mixture)} training examples to {OUTPUT_PATH}")
0049:
0050:
0051: if __name__ == "__main__":
0052:     main()

```

16. P. ml/evaluate_modernizer.py - Evaluation script

File: /Users/rutwikbhondave/Desktop/Helix AI Project/ml/evaluate_modernizer.py

Simple explanation: this code block (lines 1 to 99) mainly handles benchmark execution and pass-rate reporting. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: benchmark execution and pass-rate reporting.
- Implementation density in this block: functions=5, classes=0, endpoint decorators=0.
- Key functions to study first: load_jsonl, run_rule_pipeline, contains_all, main, run_rule_pipeline.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import argparse
0002: import asyncio
0003: import json
0004: from pathlib import Path
0005: import sys
0006:
0007: ROOT = Path(__file__).resolve().parent.parent
0008: if str(ROOT) not in sys.path:
0009:     sys.path.insert(0, str(ROOT))
0010:
0011: from agents.analyzer import AnalyzerAgent
0012: from agents.critic import CriticAgent
0013: from agents.planner import PlannerAgent
0014: from agents.suggester import SuggestionAgent
0015: from core.execution import ExecutionCore
0016: from core.ml_reasoner import MLReasoner
0017: from core.validation import ValidationCore
0018:
0019:
0020: DEFAULT_BENCHMARK = Path("ml/data/eval_benchmark.jsonl")
0021:
0022:
0023: def load_jsonl(path: Path) -> list[dict]:
0024:     rows = []
0025:     with path.open(encoding="utf-8") as handle:
0026:         for line in handle:
0027:             line = line.strip()
0028:             if line:
0029:                 rows.append(json.loads(line))
0030:     return rows
0031:
0032:
0033: async def run_rule_pipeline(code: str) -> dict:
```



```

0034:     analysis = await AnalyzerAgent().process(code)
0035:     suggestions = await SuggestionAgent().process(analysis, code)
0036:     critiques = await CriticAgent().process(suggestions)
0037:     plan = await PlannerAgent().process(suggestions, critiques)
0038:     output = ExecutionCore().apply_changes(code, plan.get("selected_plans", []))
0039:     validation = ValidationCore().validate(code, output)
0040:     return {"analysis": analysis, "plan": plan, "output": output, "validation": validation}
0041:
0042:
0043: def contains_all(text: str, expected: list[str]) -> bool:
0044:     return all(item in text for item in expected)
0045:
0046:
0047: def main() -> None:
0048:     parser = argparse.ArgumentParser()
0049:     parser.add_argument("--benchmark", default=str(DEFAULT_BENCHMARK))
0050:     parser.add_argument("--enable-ml", action="store_true")
0051:     parser.add_argument("--adapter-path", default="ml/models/nebula-modernizer-qwen25-1.5b")
0052:     parser.add_argument("--base-model", default="Qwen/Qwen2.5-Coder-7B-Instruct")
0053:     args = parser.parse_args()
0054:
0055:     benchmark = load_jsonl(Path(args.benchmark))
0056:     reasoner = MLReasoner(args.enable_ml, args.adapter_path, args.base_model)
0057:
0058:     rule_pass = 0
0059:     ml_pass = 0
0060:     rows = []
0061:     for item in benchmark:
0062:         rule_result = asyncio.run(run_rule_pipeline(item["input"]))
0063:         rule_ok = rule_result["validation"]["success"] and contains_all(rule_result["output"], item["expected_contains"])
0064:         if rule_ok:
0065:             rule_pass += 1
0066:
0067:         ml_ok = False
0068:         ml_output = ""
0069:         if args.enable_ml:
0070:             ml_result = reasoner.modernize(item["input"])
0071:             ml_output = ml_result.output
0072:             ml_ok = ml_result.available and contains_all(ml_output, item["expected_contains"])
0073:             if ml_ok:
0074:                 ml_pass += 1
0075:
0076:         rows.append(
0077:             {
0078:                 "name": item["name"],
0079:                 "rule_ok": rule_ok,
0080:                 "rule_validation": rule_result["validation"]["success"],
0081:                 "ml_ok": ml_ok,
0082:                 "rule_output": rule_result["output"],
0083:                 "ml_output": ml_output,
0084:             }
0085:         )
0086:
0087:     print(json.dumps(
0088:         {
0089:             "benchmark_size": len(benchmark),
0090:             "rule_pass": rule_pass,
0091:             "ml_pass": ml_pass if args.enable_ml else None,
0092:             "details": rows,
0093:         },
0094:         indent=2,
0095:     ))
0096:
0097:
0098: if __name__ == "__main__":
0099:     main()

```

17. Q. tests/test_pipeline.py - Pipeline tests

File: /Users/rutwikbhondave/Desktop/Helix AI Project/tests/test_pipeline.py

Simple explanation: this code block (lines 1 to 100) mainly handles automated correctness checks and regression safety. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: automated correctness checks and regression safety.
- Implementation density in this block: functions=6, classes=1, endpoint decorators=0.
- Key functions to study first: `legacy_scan`, `legacy_formatter`, `test_analyzer_detects_legacy_python_patterns`, `test_pipeline_modernizes_legacy_code`, `test_validation_rejects_leftover_legacy_constructs`.
- Key classes in this block: `PipelineTests`.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import asyncio
0002: import unittest
0003:
0004: from agents.analyzer import AnalyzerAgent
0005: from agents.critic import CriticAgent
0006: from agents.planner import PlannerAgent
0007: from agents.suggester import SuggestionAgent
0008: from core.execution import ExecutionCore
0009: from core.validation import ValidationCore
0010:
0011:
0012: LEGACY_SAMPLE = """def legacy_scan(values):
0013:     print "starting migration"
0014:     for i in xrange(len(values)):
0015:         if values.has_key(i):
0016:             print values[i]
0017:     return raw_input("done? ")
0018: """
0019:
0020: VERY_OLD_SAMPLE = """def legacy_formatter(value, args):
0021:     print `value`
0022:     return apply(str, args)
0023: """
0024:
0025:
0026: class PipelineTests(unittest.TestCase):
0027:     def test_analyzer_detects_legacy_python_patterns(self):
0028:         analysis = asyncio.run(AnalyzerAgent().process(LEGACY_SAMPLE))
```

```

0029:
0030:         self.assertTrue(analysis["success"])
0031:         self.assertEqual(analysis["probable_source_version"], "Python 2.x / early legacy Python")
0032:         self.assertGreaterEqual(len(analysis["legacy_issues"]), 4)
0033:         issue_ids = {issue["id"] for issue in analysis["legacy_issues"]}
0034:         self.assertIn("print_statement", issue_ids)
0035:         self.assertIn("xrange_usage", issue_ids)
0036:         self.assertIn("has_key_usage", issue_ids)
0037:         self.assertIn("raw_input_usage", issue_ids)
0038:
0039:     def test_pipeline_modernizes_legacy_code(self):
0040:         analyzer = AnalyzerAgent()
0041:         suggester = SuggestionAgent()
0042:         critic = CriticAgent()
0043:         planner = PlannerAgent()
0044:         executor = ExecutionCore()
0045:         validator = ValidationCore()
0046:
0047:         analysis = asyncio.run(analyzer.process(LEGACY_SAMPLE))
0048:         suggestions = asyncio.run(suggester.process(analysis, LEGACY_SAMPLE))
0049:         critiques = asyncio.run(critic.process(suggestions))
0050:         plan = asyncio.run(planner.process(suggestions, critiques))
0051:
0052:         modernized = executor.apply_changes(LEGACY_SAMPLE, plan["selected_plans"])
0053:         validation = validator.validate(LEGACY_SAMPLE, modernized)
0054:
0055:         self.assertIn("print(\"starting migration\")", modernized)
0056:         self.assertIn("range(len(values))", modernized)
0057:         self.assertIn("i in values", modernized)
0058:         self.assertIn("input(\"done? \")", modernized)
0059:         self.assertTrue(validation["success"])
0060:
0061:     def test_validation_rejects_leftover_legacy_constructs(self):
0062:         validator = ValidationCore()
0063:         result = validator.validate(
0064:             "for i in xrange(3):\n    print i\n",
0065:             "for i in xrange(3):\n    print(i)\n",
0066:         )
0067:
0068:         self.assertFalse(result["success"])
0069:         self.assertEqual(result["stage"], "LINT")
0070:         self.assertIn("xrange", result["error"])
0071:
0072:     def test_pipeline_handles_very_old_legacy_patterns(self):
0073:         analyzer = AnalyzerAgent()
0074:         suggester = SuggestionAgent()
0075:         critic = CriticAgent()
0076:         planner = PlannerAgent()
0077:         executor = ExecutionCore()
0078:         validator = ValidationCore()
0079:
0080:         analysis = asyncio.run(analyzer.process(VERY_OLD_SAMPLE))
0081:         self.assertTrue(analysis["success"])
0082:         self.assertEqual(analysis["probable_source_version"], "Python 1.x / early legacy Python")
0083:         issue_ids = {issue["id"] for issue in analysis["legacy_issues"]}
0084:         self.assertIn("backtick_repr", issue_ids)
0085:         self.assertIn("apply_builtin", issue_ids)
0086:
0087:         suggestions = asyncio.run(suggester.process(analysis, VERY_OLD_SAMPLE))
0088:         critiques = asyncio.run(critic.process(suggestions))
0089:         plan = asyncio.run(planner.process(suggestions, critiques))
0090:
0091:         modernized = executor.apply_changes(VERY_OLD_SAMPLE, plan["selected_plans"])
0092:         validation = validator.validate(VERY_OLD_SAMPLE, modernized)
0093:
0094:         self.assertIn("print(repr(value))", modernized)
0095:         self.assertIn("str(*args)", modernized)
0096:         self.assertTrue(validation["success"])
0097:
0098:

```

```
0099: if __name__ == "__main__":  
0100:     unittest.main()
```

18. R. tests/test_api.py - API tests

File: /Users/rutwikbhondave/Desktop/Helix AI Project/tests/test_api.py

Simple explanation: this code block (lines 1 to 67) mainly handles automated correctness checks and regression safety. It is part of the practical path from user input to safe modernization output.

How this code works (technical):

- Technical role: automated correctness checks and regression safety.
- Implementation density in this block: functions=5, classes=1, endpoint decorators=0.
- Key functions to study first: setUpClass, test_analyze_endpoint_returns_plan_and_ml_status, test_refactor_endpoint_returns_diff_and_validation, test_health_and_model_status_endpoints, test_runs_endpoints_return_stored_history.
- Key classes in this block: ApiTests.
- Validation-related checks are present, so this block contributes to safety gating.

Risk and quality notes:

- If this block changes without tests, behavior regressions may appear in migration output.
- Edge cases may be hidden when input format is unexpected.
- Whenever logic changes here, rerun pipeline tests and benchmark to verify no silent breakage.

How to explain this in viva:

- Viva tip: explain this block first in one simple sentence, then mention one metric and one test that validates it.
- Viva tip: clearly separate what is deterministic here versus what is only advisory.
- Viva tip: mention one limitation and a concrete improvement plan for this exact module.

Code excerpt:

```
0001: import unittest
0002:
0003: from fastapi.testclient import TestClient
0004:
0005: from main import app
0006:
0007:
0008: LEGACY_SAMPLE = """try:
0009:     risky()
0010: except ValueError, exc:
0011:     print exc
0012: """
0013:
0014:
0015: class ApiTests(unittest.TestCase):
0016:     @classmethod
0017:     def setUpClass(cls):
0018:         cls.client = TestClient(app)
0019:
0020:     def test_analyze_endpoint_returns_plan_and_ml_status(self):
0021:         response = self.client.post("/analyze", json={"code": LEGACY_SAMPLE})
0022:         self.assertEqual(response.status_code, 200)
0023:
0024:         payload = response.json()
0025:         self.assertIn("analysis", payload)
0026:         self.assertIn("plan", payload)
0027:         self.assertIn("ml_reasoner", payload)
0028:         self.assertTrue(payload["analysis"]["success"])
```

```

0029:         self.assertEqual(payload["analysis"]["probable_source_version"], "Python 2.x / early legac
0030:
0031:     def test_refactor_endpoint_returns_diff_and_validation(self):
0032:         response = self.client.post("/refactor", json={"code": LEGACY_SAMPLE})
0033:         self.assertEqual(response.status_code, 200)
0034:
0035:         payload = response.json()
0036:         self.assertIn("new_code", payload)
0037:         self.assertIn("diff", payload)
0038:         self.assertIn("validation", payload)
0039:         self.assertIn("except ValueError as exc", payload["new_code"])
0040:         self.assertIn("print(exc)", payload["new_code"])
0041:         self.assertTrue(payload["validation"]["success"])
0042:
0043:     def test_health_and_model_status_endpoints(self):
0044:         health = self.client.get("/health")
0045:         model_status = self.client.get("/model-status")
0046:
0047:         self.assertEqual(health.status_code, 200)
0048:         self.assertEqual(model_status.status_code, 200)
0049:         self.assertEqual(health.json()["status"], "ok")
0050:         self.assertIn("available", model_status.json())
0051:
0052:     def test_runs_endpoints_return_stored_history(self):
0053:         refactor = self.client.post("/refactor", json={"code": LEGACY_SAMPLE})
0054:         self.assertEqual(refactor.status_code, 200)
0055:         run_id = refactor.json()["run_id"]
0056:
0057:         runs = self.client.get("/runs?limit=5")
0058:         single = self.client.get(f"/runs/{run_id}")
0059:
0060:         self.assertEqual(runs.status_code, 200)
0061:         self.assertEqual(single.status_code, 200)
0062:         self.assertTrue(any(item["run_id"] == run_id for item in runs.json()["runs"]))
0063:         self.assertEqual(single.json()["run_id"], run_id)
0064:
0065:
0066: if __name__ == "__main__":
0067:     unittest.main()

```

Chapter 11: API Reference and Example Payloads

Method	Route	Purpose
GET	/health	Basic system health and ML status summary
GET	/model-status	Detailed model enabled/available/error state
POST	/analyze	Legacy detection, suggestions, critique, and plan
POST	/refactor	Full modernization run with validation and diff
GET	/runs	Recent run summaries
GET	/runs/{run_id}	Full run detail
GET	/app	Inner modernization UI page
GET	/	Landing page

Example analyze request/response skeleton:

```
POST /analyze
{
  "code": "for i in xrange(3):\n    print i"
}

{
  "analysis": { "...": "..." },
  "suggestions": [ ... ],
  "critiques": { "critiques": [ ... ] },
  "plan": { "selected_plans": [ ... ] },
  "ml_reasoner": { "enabled": false, "available": false }
}
```

Example refactor response skeleton:

```
{
  "run_id": "abc123...",
  "original_code": "...",
  "new_code": "...",
  "analysis": { ... },
  "suggestions": [ ... ],
  "plan": { "selected_plans": [ ... ] },
  "validation": { "success": true, "stage": "COMPLETE" },
  "logs": [ ... ],
  "report": "# Legacy Python Modernization Report ...",
  "diff": "--- legacy.py\n+++ modernized.py\n...",
  "ml_reasoner": { ... }
}
```

Chapter 12: Teaching Plan (How to Learn This Project Step-by-Step)

1. Phase 1 - Basics (1 to 2 days)

- Python syntax basics
- Functions and loops
- Files and modules
- Basic regex

Deliverable for this phase: write one-page notes explaining what you learned and one small code experiment you completed.

2. Phase 2 - Backend Core (2 to 3 days)

- Understand FastAPI routes
- Read analyzer/suggester/critic/planner
- Trace refactor endpoint flow

Deliverable for this phase: write one-page notes explaining what you learned and one small code experiment you completed.

3. Phase 3 - Frontend Flow (1 to 2 days)

- Understand state object in app.js
- Trace button action flow
- Inspect tab rendering functions

Deliverable for this phase: write one-page notes explaining what you learned and one small code experiment you completed.

4. Phase 4 - Validation and Safety (1 day)

- Study validation stages
- Run failing/blocked examples
- Observe rollback behavior

Deliverable for this phase: write one-page notes explaining what you learned and one small code experiment you completed.

5. Phase 5 - ML Pipeline (2 to 4 days)

- Read dataset curation scripts
- Run Colab training flow
- Integrate adapter and verify model-status

Deliverable for this phase: write one-page notes explaining what you learned and one small code experiment you completed.

6. Phase 6 - Testing and Defense (1 to 2 days)

- Run all tests
- Run benchmark

- Prepare viva Q&A with examples

Deliverable for this phase: write one-page notes explaining what you learned and one small code experiment you completed.

Chapter 13: Viva Preparation Bank (Question and Answer)

1. Q: What is Helix AI?

A: A specialized system that modernizes legacy Python using deterministic transforms, validation gates, and optional ML assistance.

2. Q: Why not only LLM?

A: Because deterministic migration and validation are critical for trust and reproducibility.

3. Q: How many stages are in your pipeline?

A: Analyzer, Suggester, Critic, Planner, Execution, Validation, and Report.

4. Q: How do you detect legacy code?

A: Using explicit regex-based rules and source-version hints in analyzer.

5. Q: How do you prevent unsafe migrations?

A: Risk modes, critic warnings, staged validation, and rollback on failure.

6. Q: What is your scoring formula?

A: Planner final score combines critic safety score with priority-based boost.

7. Q: How do you validate behavior?

A: Function name preservation and semantic warning checks are included.

8. Q: Where is run history stored?

A: Append-only JSONL managed by RunStore.

9. Q: How is frontend workflow organized?

A: Single-page state-driven flow with analyze/review/modernize/validate phases.

10. Q: How do you train ML model?

A: Prepare dataset mixture, run LoRA fine-tuning in Colab, export adapter, integrate via env vars.

11. Q: What are your current metrics?

A: Unit tests: 10 (pass), benchmark: 4/4.

12. Q: Can it handle Python 1 style?

A: Selected early-legacy constructs like backtick repr/apply are handled via explicit rules.

13. Q: What is a key limitation?

A: Not every historical edge case is auto-fixed; high-risk or unsupported cases need manual review.

14. Q: What is your roadmap?

A: More pattern coverage, larger benchmarks, stronger ML calibration, and CI quality gates.

15. Q: Why is this project practical?

A: It is already runnable end-to-end with UI, APIs, tests, and report outputs.

Chapter 14: Final Conclusion

This report presents the Helix AI project from scratch to full working state in one place. It includes concepts, implementation details, code evidence, evaluation snapshot, and learning guidance.

The strongest engineering characteristic of the current version is the safety-first deterministic pipeline with transparent diff/report outputs. This makes the tool suitable for educational defense and practical migration demos.

The strongest future opportunity is scaling pattern coverage and benchmark depth while preserving deterministic trust. The current architecture already supports that roadmap without redesigning from zero.

Use this report as your master reference for submission, viva preparation, and onboarding teammates who need to understand the system deeply but in simple language first.